Laboratory 2
Wind Turbine model

Let us build a model simulating how a maintenance center services wind turbines. There are 10 turbines, placed randomly in continuous space, that require maintenance.

There are two types of maintenance activities:

- Periodic maintenance:
  - Should be done once every two weeks.
  - Maintenance crew reaches the turbine by truck.
  - Service time is 10 hours.
  - While waiting for a crew to perform periodic maintenance, failure may occur once in a week.
- Emergency failure repairs:
  - Mean time between failures equals 250 hours.
  - Maintenance crew reaches the turbine by helicopter.
  - Service time is uniformly distributed from 10 to 20 hours.

Wind turbines are serviced by one maintenance center. Center has a fleet of 2 helicopters and 5 trucks.

Optionally you can consider activity-based costing, adding costs per crew, adding part replacement logic with parts of different costs, etc

Phase 1. Creating the agent population

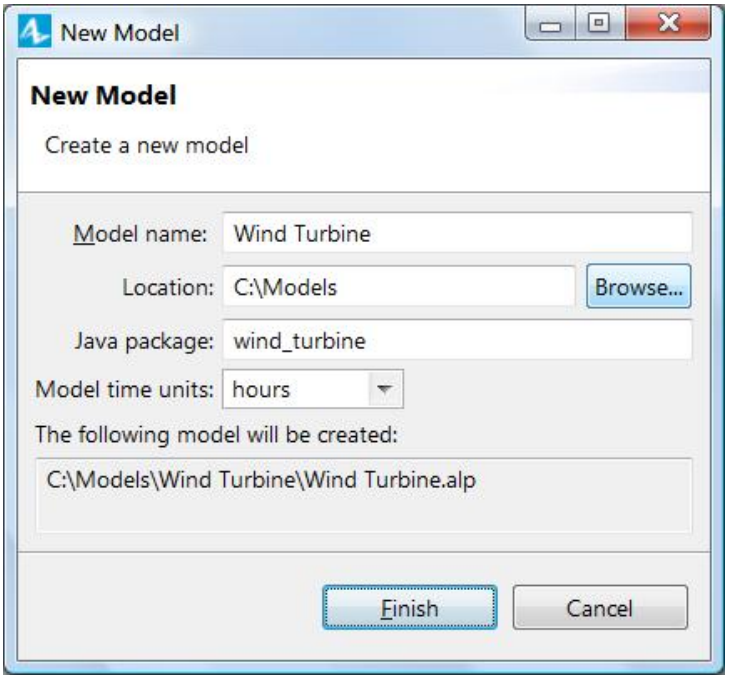
Agents are the main building blocks of this AnyLogic model. Agent is a unit of model design that can have behavior, memory (history), timing, contacts, etc. You can define as many agent types in your model as you need. We will create and define a single agent, an agent type and three agent populations in this model.

The agent internal state and behavior can be implemented in a number of ways. The state of the agent can be represented by a number of variables, by the statechart state, etc. The behavior can be so to say passive (e.g. there are agents that only react to message arrivals or to function calls and do not have their own timing), or active, when internal dynamics (timeouts or system dynamics processes) of the agent causes it to act. In the latter case agents most probably would have event and/or statechart objects inside.

In our model we will define agents' behavior using parameters, variables, collections, functions, events, option lists and also actioncharts and statecharts.

1. Create a new model by selecting **File** > **New** > **Model** from AnyLogic's main menu. The **New Model** wizard will open.

2. Specify the name of the model. Type *Wind Turbine* in the **Model name** edit box.



3. Specify the location where you want to store your model files. Browse for the existing folder using the **Browse** button, or type the name of the folder you want to create in the **Location** edit box.

4. Select the **Model time units**: *hours*.

5. Click **Finish** to complete the process.

   New model is created. It already has one agent type called *Main* and experiment called *Simulation*.

   In the center of the workspace you will see the graphical editor. It shows the diagram of the *Main* agent type. The blue frame on the diagram shows borders of the presentation window (the presentation window settings are located in the **Window** section of *Simulation* experiment properties).

   To the left of the graphical editor you can see the **Projects** view and the **Palette** view sharing the same area. The **Projects** view provides access to AnyLogic models currently opened in the workspace. The workspace tree provides easy navigation throughout the models. The **Palette** view contains all graphical elements you can add onto the graphical editor of your agent just by drag and drop. Model elements are grouped by categories in a number of stencils (palettes).

   On the right side of the workspace you can see the **Properties** view. The **Properties** view is used to view and modify the properties of a currently selected model element(s). When you select something – e.g., in the **Projects** view or in the graphical editor – the **Properties** view displays the properties of the selection.

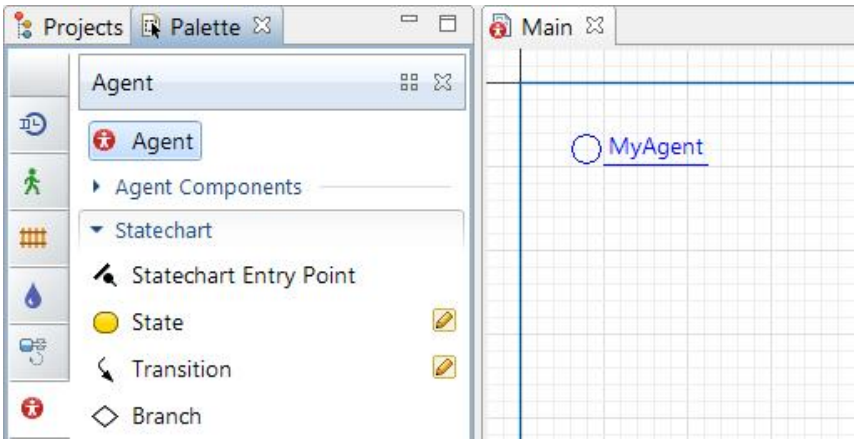
   Now we can continue developing our model created by AnyLogic model creation wizard.

   First we need to create a single agent - the maintenance center, then an empty population for maintenance transport, three populations for the wind turbines, trucks and helicopters,

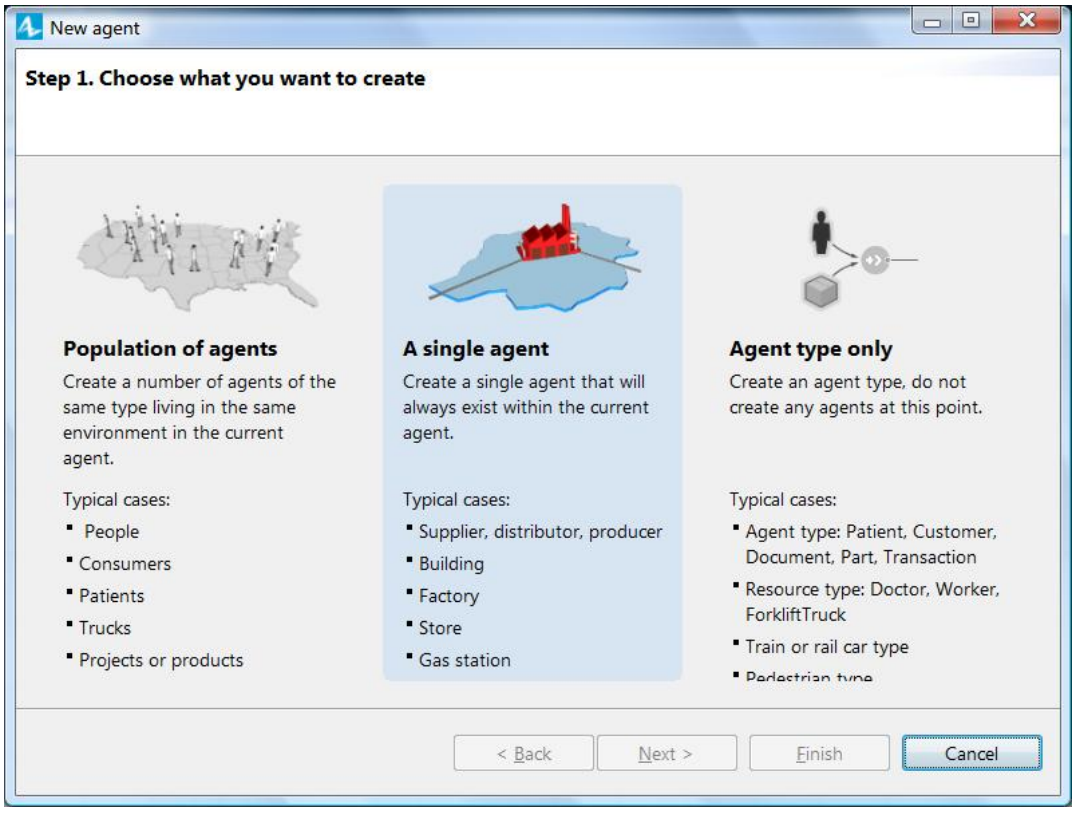
and an agent type to represent service requests. Then we will be able to define the processes inside the types of agents - on their diagrams.

6. In the **Palette** view, hover your mouse over the vertical navigation bar and select **Agent** there to open the **Agent** palette.

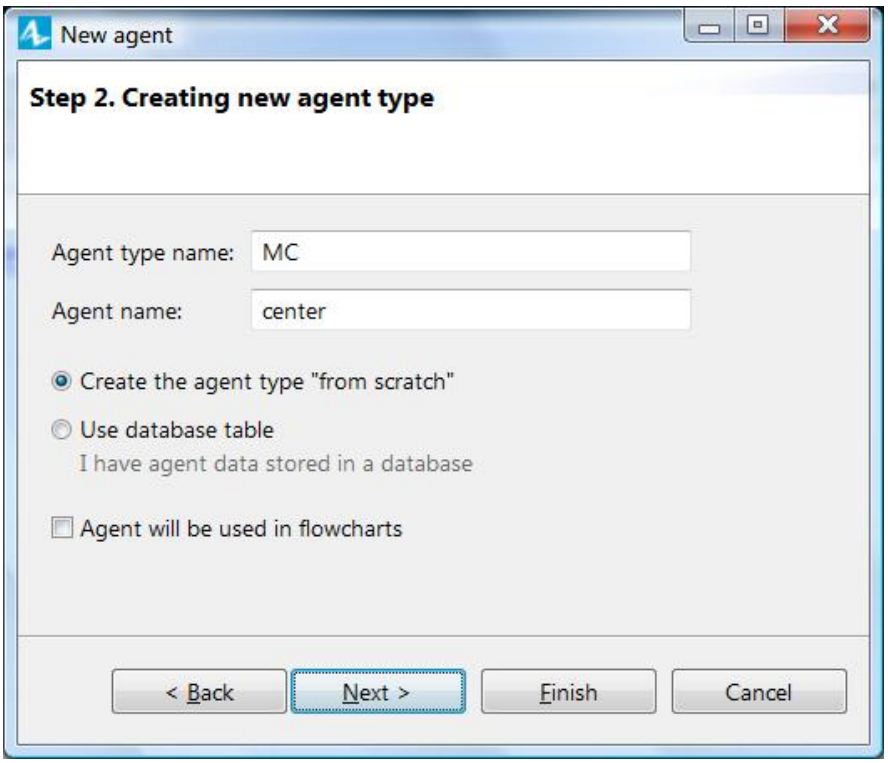
7. Drag the **Agent** element from the **Agent** palette into *Main* diagram. The **New agent** wizard will pop up automatically.



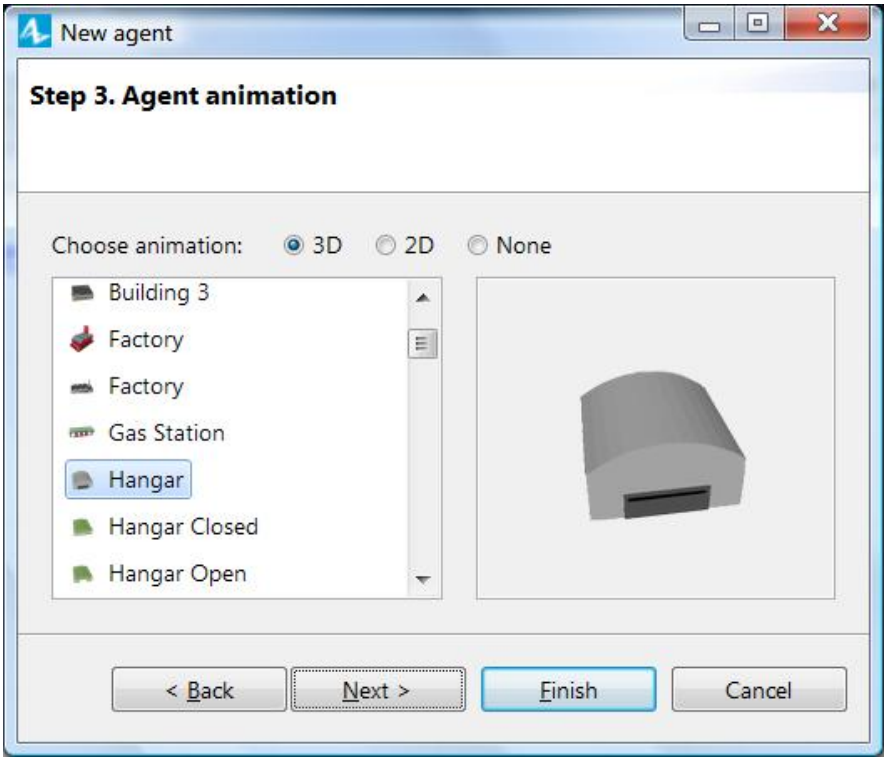
8. Select **A single agent** option on Step 1. We need only one maintenance center that will send transport to turbines for scheduled maintenance and any failures/repairs.



9. We are creating a new type of agent and do not need to use any templates. Specify the **Agent type name**: *MC* and the **Agent name**: *center*. Click **Next** to proceed.

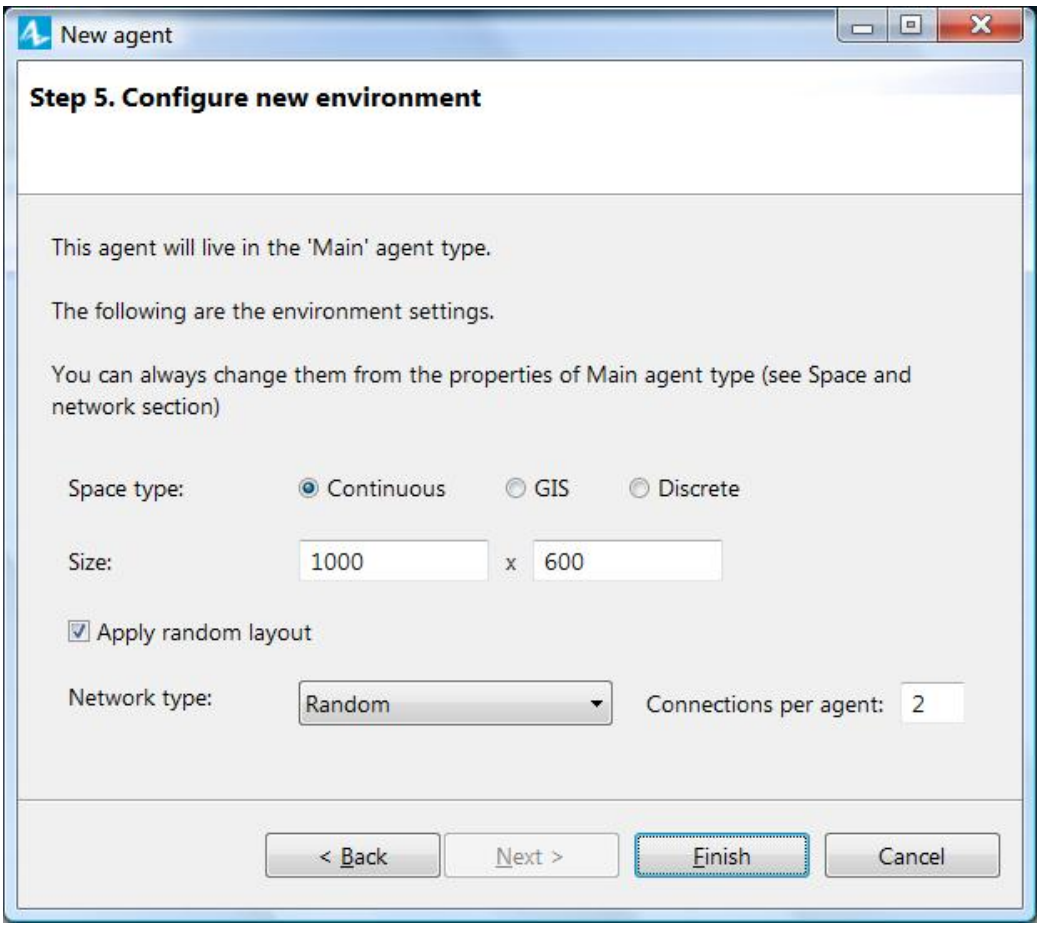


10. Choose the **3D** animation figure *Hangar* from the **Buildings** section and click **Next**.



11. Skip Step 4 where you can add parameters, simply click **Next**.

12. We want all our agents to live in a continuous space, 1000x600, in a random network with a random layout. The agent populations that we will add on *Main* will also live in the same environment.



13. Click **Finish**.

    Now you can find the new agent type *MC* in the model tree on the same level as *Main* and the agent *center* itself under *Main* > *Agents*.

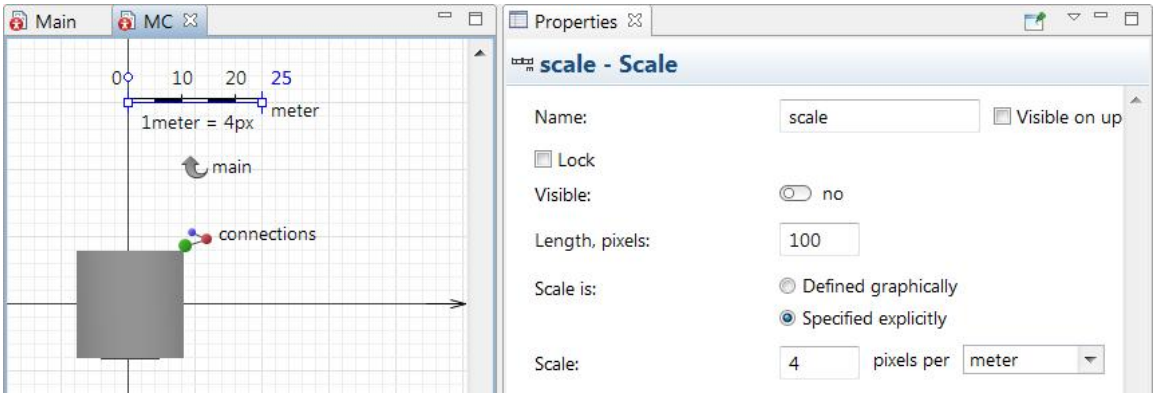
    The agent animation is shown on the agent type diagram as well as on *Main*. We need to change the scale of this agent's animation.

    We cannot model the real-size objects in computer simulation, so we use the scale to define the approximate correspondence between the objects being animated. We will set the same scale for all our agents.

14. Double-click *MC* in the model tree and pan the agent's diagram down.

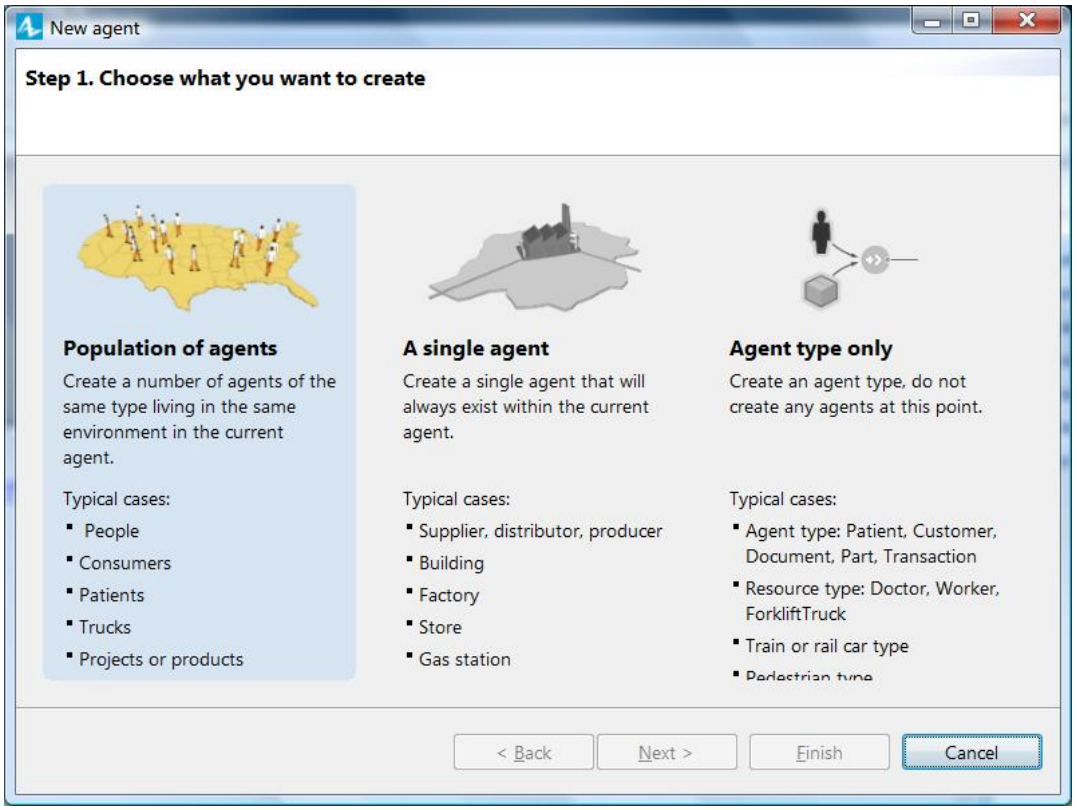
15. Above the X axis, you will find the **Scale** element. Open its **Properties** view.

16. Set the **Scale is** property to be *Specified explicitly*. Then specify the **Scale**: *4 pixels per meter*.

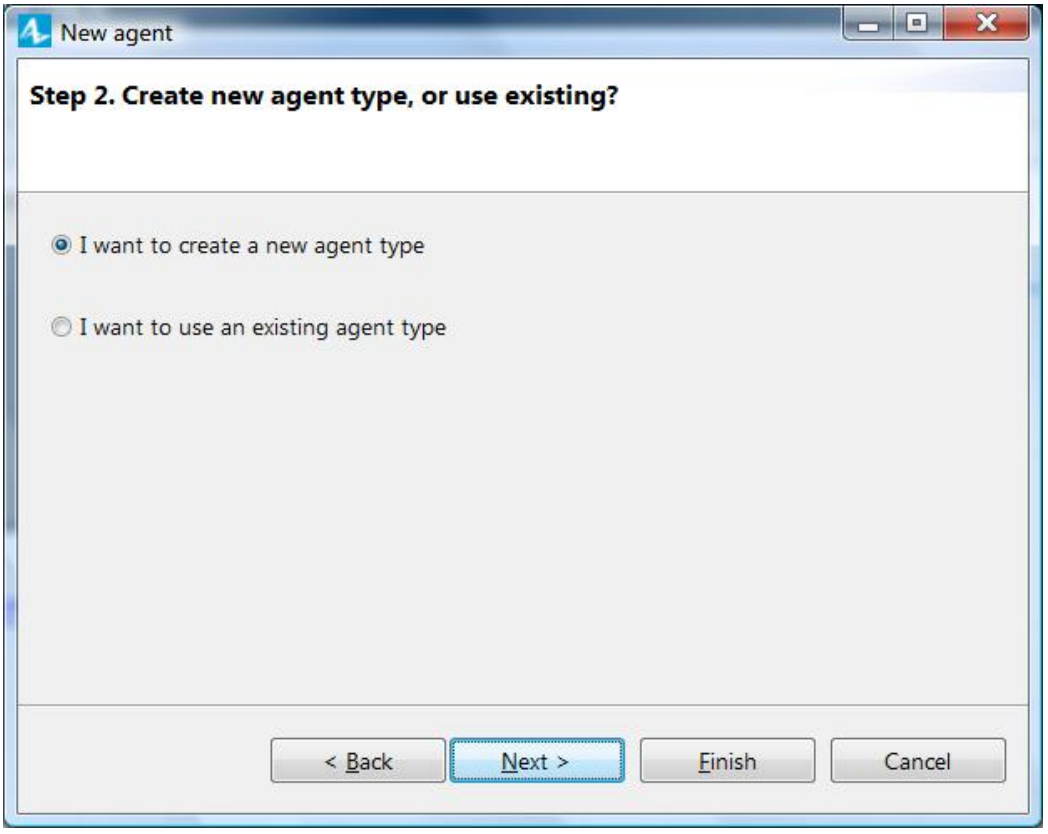


17. Drag the **Agent** element from the **Agent** palette onto *Main* diagram.

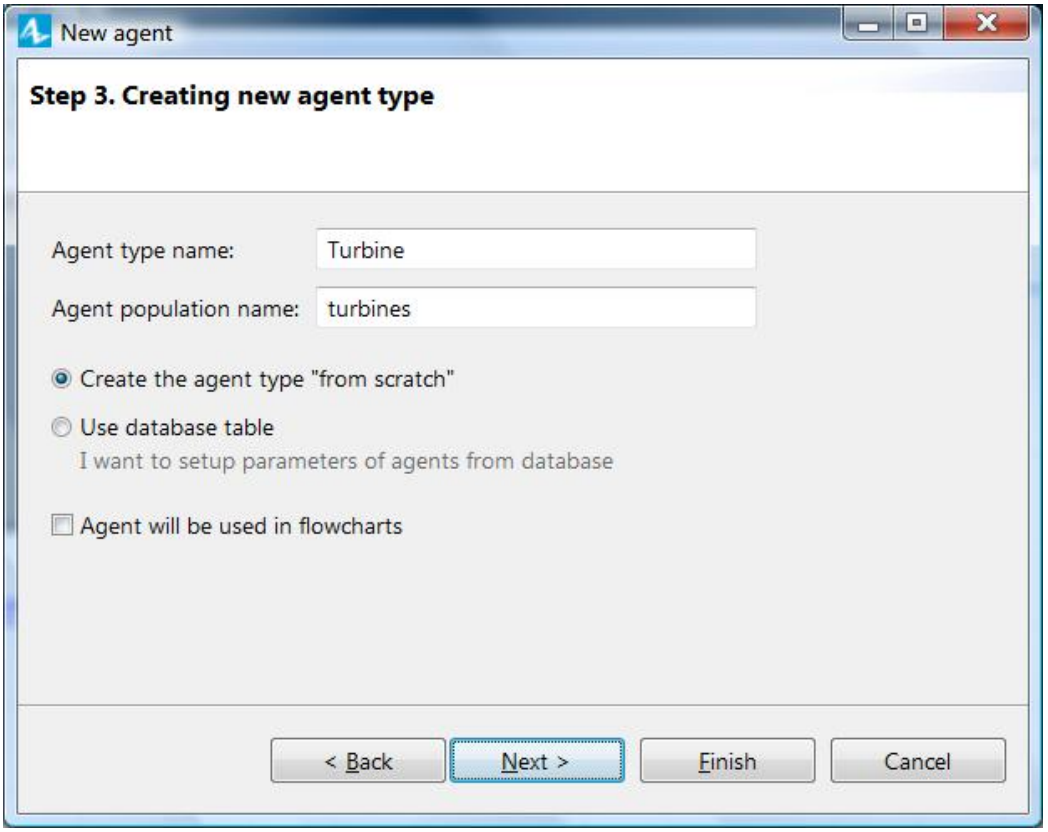
18. This time, select **Population of agents**.



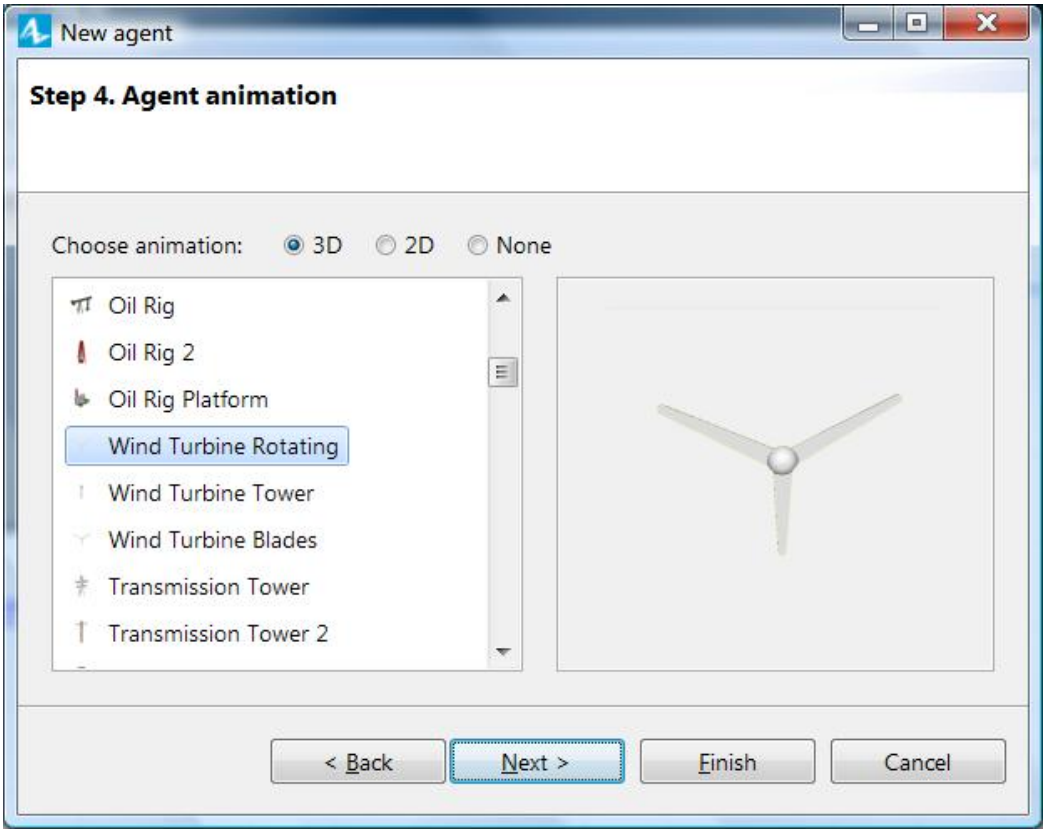
19. Click **Next** on Step 2 (the default setting **I want to create a new agent type**).



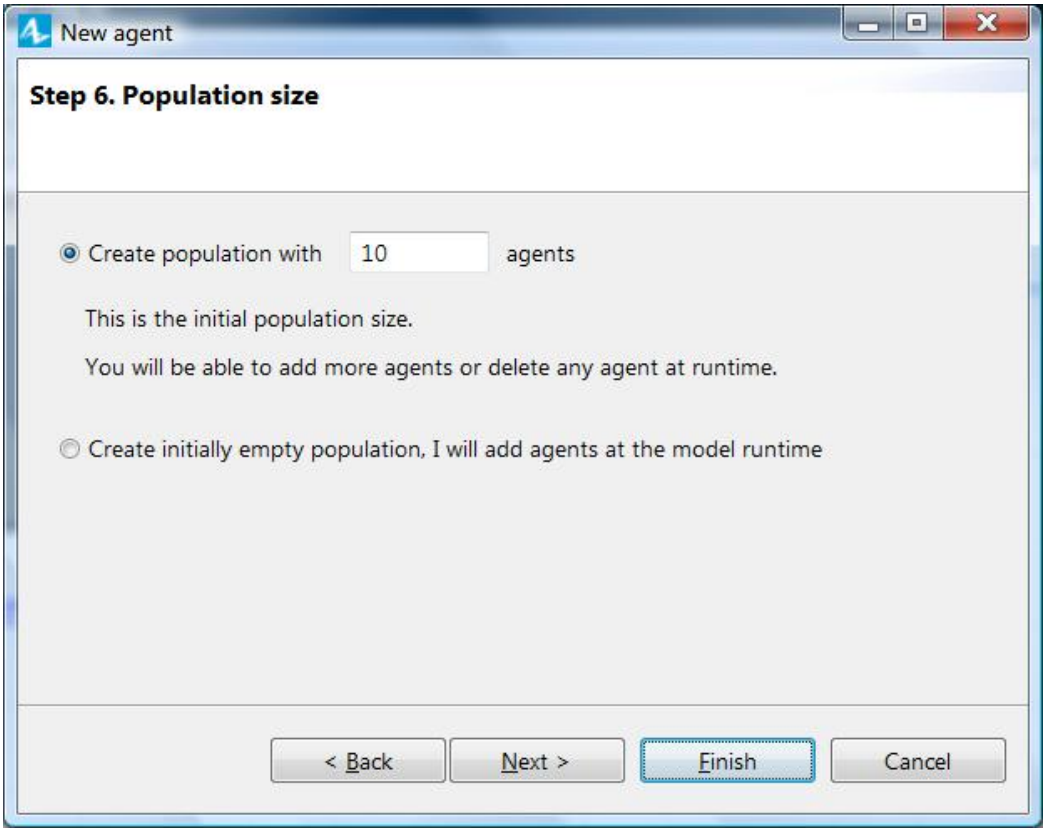
20. On the next wizard page type the name of the new type: *Turbine* and agent population name will autofill as *turbines*. Click **Next**.



21. Choose **3D** animation figure *Wind Turbine Rotating* from the **Energy** section of the list.



22. We do not need to add here any parameters, so skip that Step 5. On Step 6, set up to create *10* agents in this population.



23. You can click **Finish** on step 6. Step 7, **Configure new environment**, is prefilled with the correct settings because we are adding the new population into *Main* where there already is a configured environment.

24. Open the **Turbine** agent diagram and set the agent's **Scale** to *4 px per meter* the same way we did for the **MC** agent.

25. Drag the **Agent** element from the **Agent** palette onto *Main* diagram.

26. One more time select **Population of agents**. Click **Next** on Step 2 (**I want to create a new agent type**). On the next wizard page (step 3) define the **Agent type name**: *Transport*, and the **Agent population name**: *transport*. Click **Next**.

27. This is an empty population that does not need animation. We will use this agent type to define the logic of our model. Select **None** for animation type, skip the parameters step, choose to **Create initially empty population** and click **Finish**.

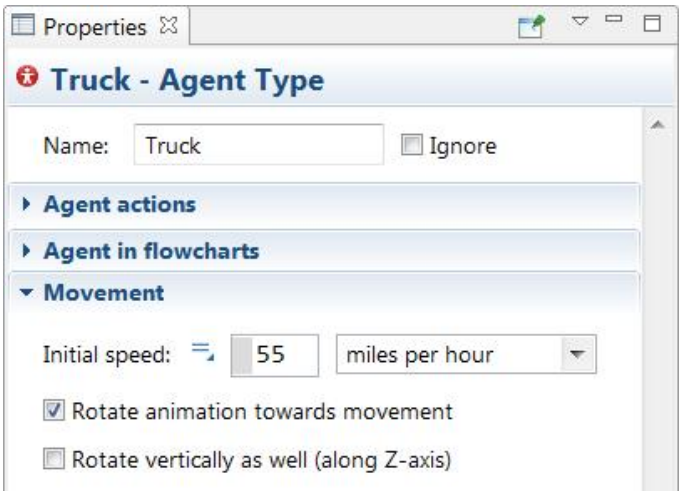
28. After the type is created, set its **Scale** to *4 px per meter*.

29. Drag the **Agent** element from the **Agent** palette onto *Main* diagram.

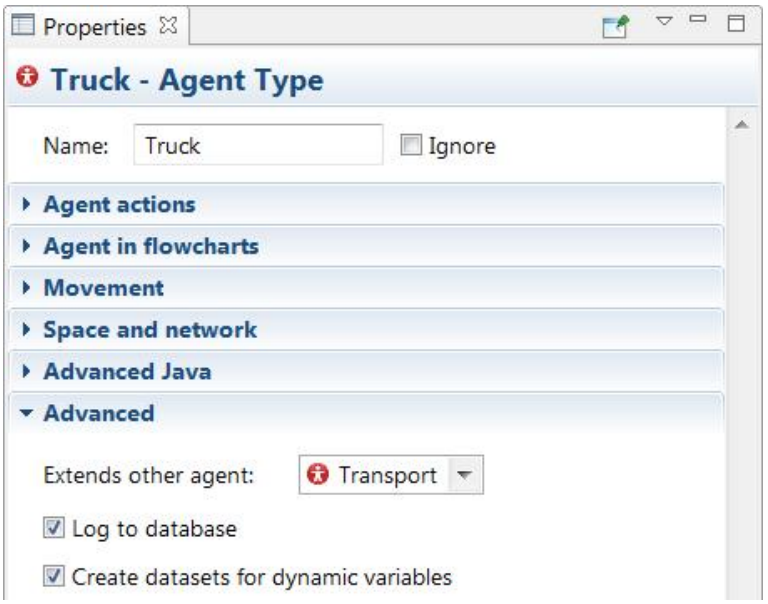
30. Again, select **Population of agents**. Click **Next** on Step 2 (**I want to create a new agent type**). On the next wizard page (step 3) define the **Agent type name**: *Truck*, and the agent population name will autofill as *trucks*. Click **Next**.

31. Select the *Lorry* **3D** animation figure from the section **Road transport** on the next step. We do not need to add any parameters here in the wizard. There are 5 agents in this population and the environment settings should be prefilled as we set them up the first time.

32. After you create the *Truck* agent type, select it the model tree and go to its **Properties** view. First expand the **Movement** section. We suppose that the average truck's speed equals *55 miles per hour*.



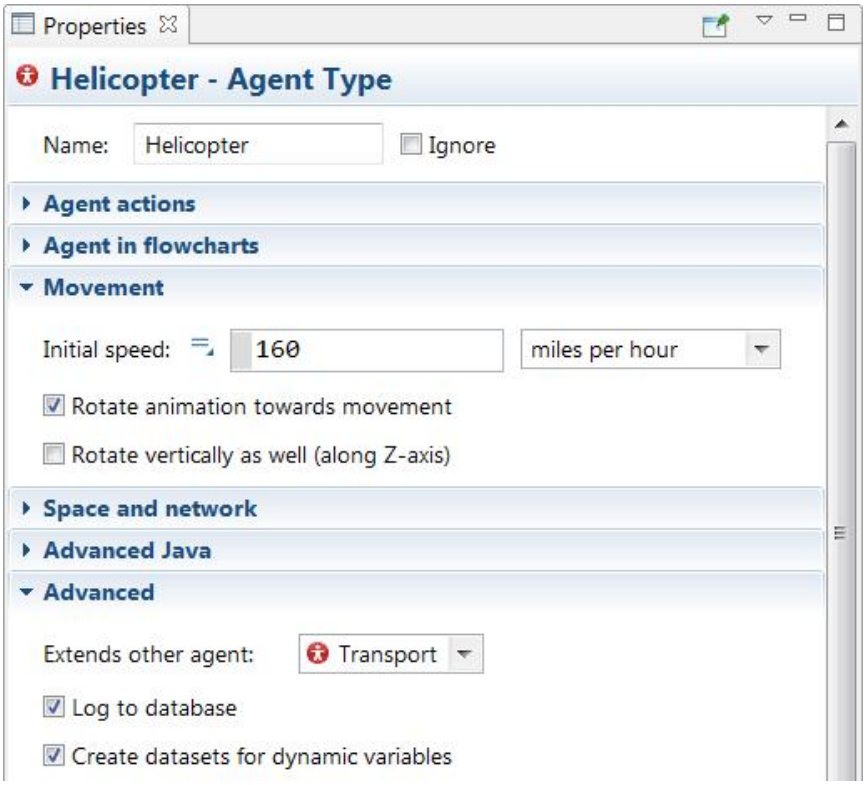
33. Expand the **Advanced** section and make *Truck* inherit *Transport* configuration by setting up the parameter **Extends other agent**: *Transport*.



34. Now *Truck* agent is fully set up. We need to add one more population for helicopters. Go back to *Main* and drag the **Agent** element from the **Agent** palette again and select **Population** on the first step.

35. Type the new agent type name *Helicopter* and the population name *helicopters* will autofill. You can find the *Helicopter* **3D** animation figure in the section **Military**. We want to use 2 helicopters for the turbine maintenance. This population lives in the same environment.

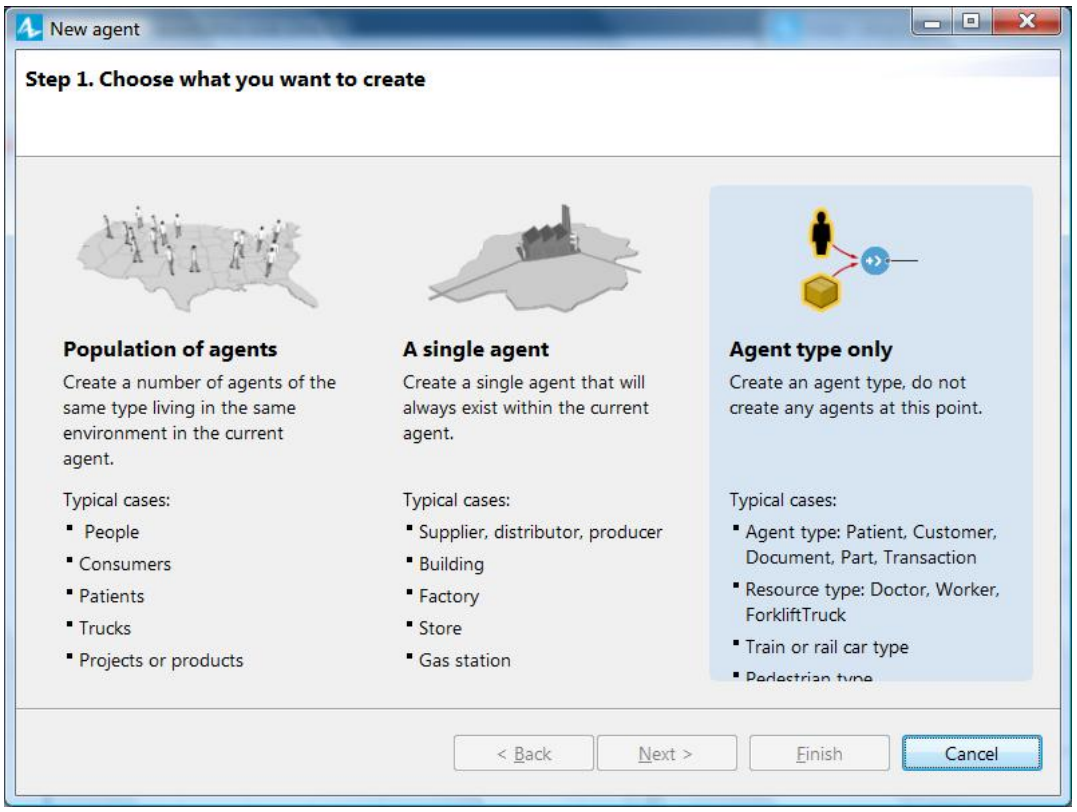
36. *Helicopter* also **Extends other agent**: *Transport*. We suppose that helicopters move with the speed *160 mph*.



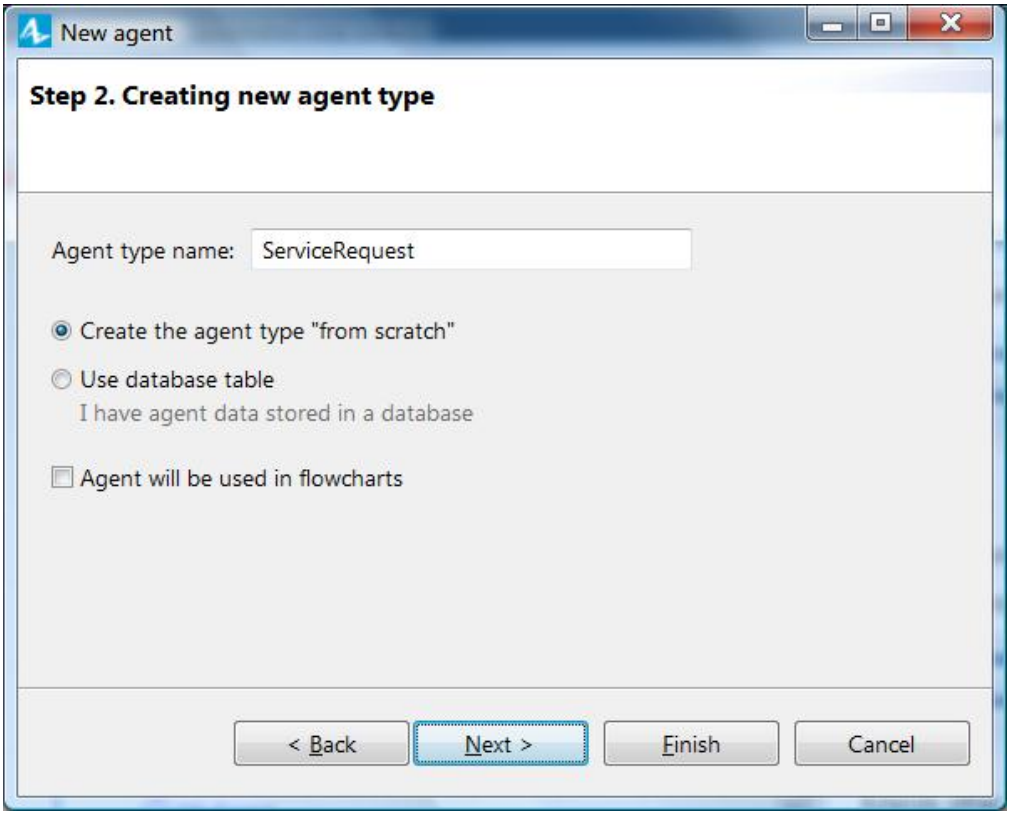
37. As you noticed, we are not changing the **Truck**'s and **Helicopter**'s scale. You can check the **Scale** element of any of those agent types: there you will see that it inherits the scale from the **Transport** agent.

38. Drag the **Agent** element from the **Agent** palette into *Main* diagram.

39. Select **Agent type only**. We need to add an agent type to define logic of the model with the parameters we will set up on its diagram.

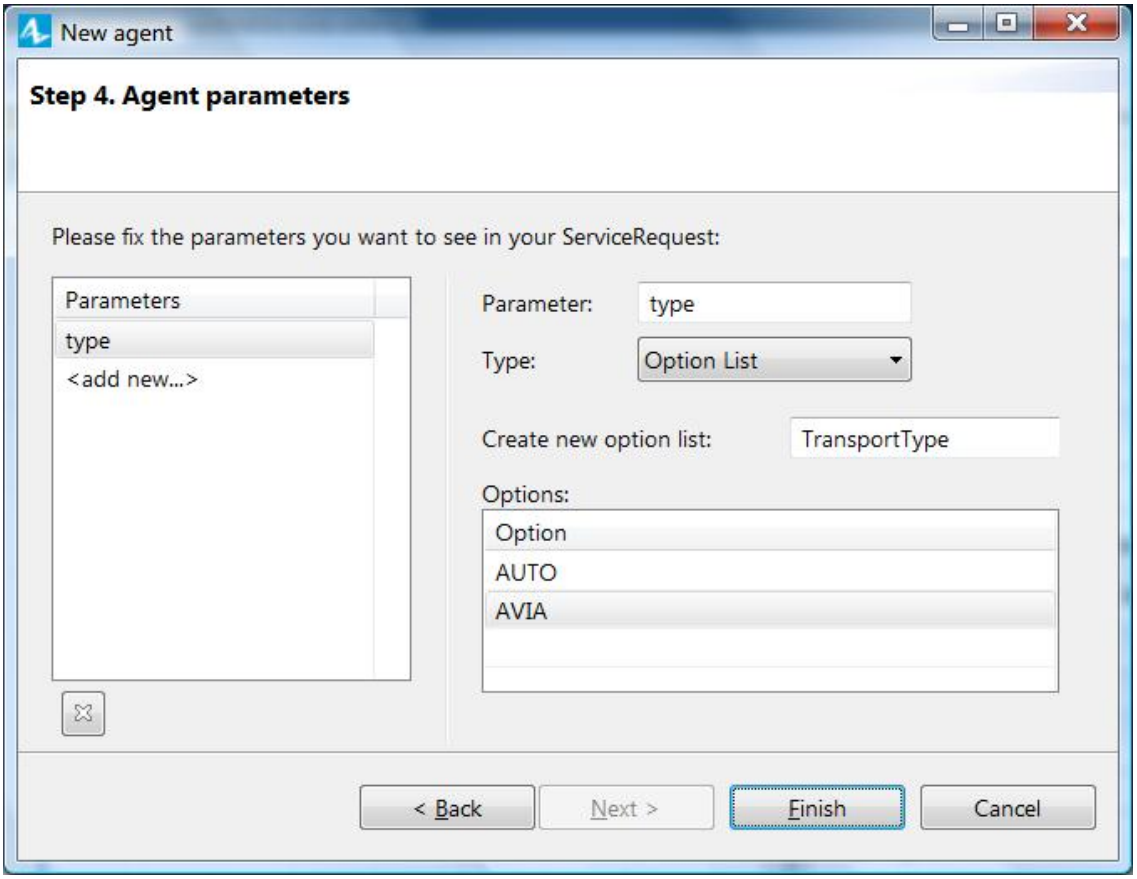


40. Type its name *ServiceRequest* in the corresponding field and click **Next**. Select **None** for the animation type and proceed to the next step.

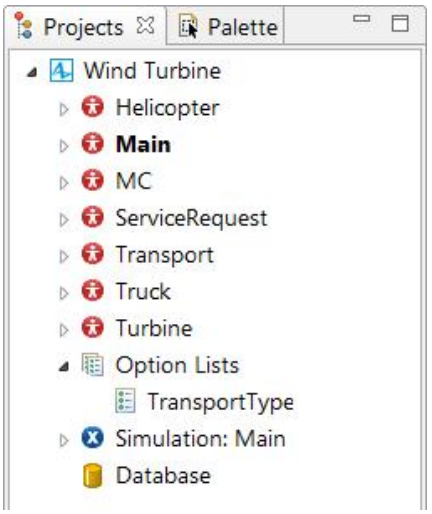


41. On step 4 we will create a parameter and an option list. **Option List** is the element for defining agent attributes that have limited choice of alternative options. In our case, we

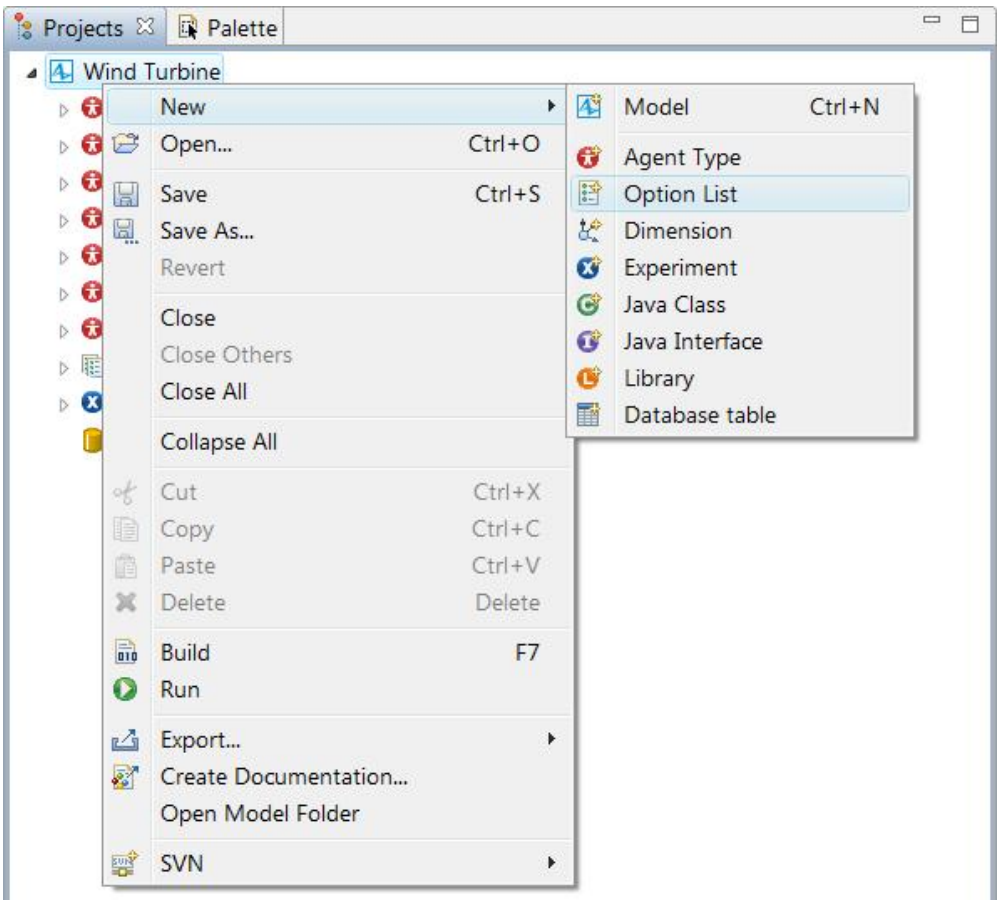
need to differentiate between trucks and helicopters which are both used by the maintenance center and respond to the service requests. The parameter is called *type* and its type is the **Option List**. Since there are no option lists in the model yet, the wizard suggests that you **Create new option list**: *TransportType*. Type the options in the table, one in a line: *AUTO*, *AVIA*.



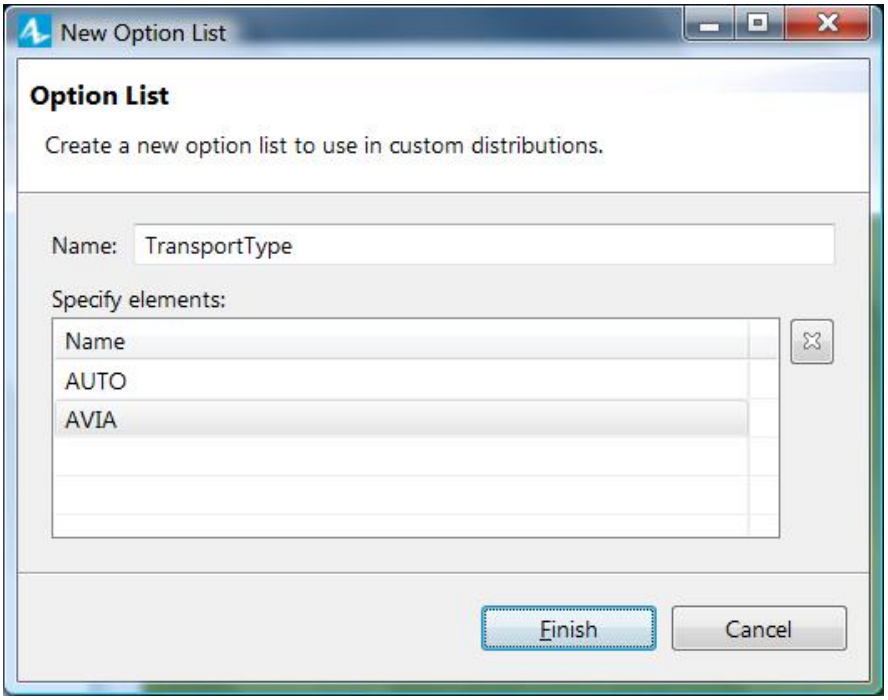
42. Click **Finish**. The option lists do not have any graphical icons that you can click to select them in the graphical editor. You can find **Option Lists** in the model elements tree:



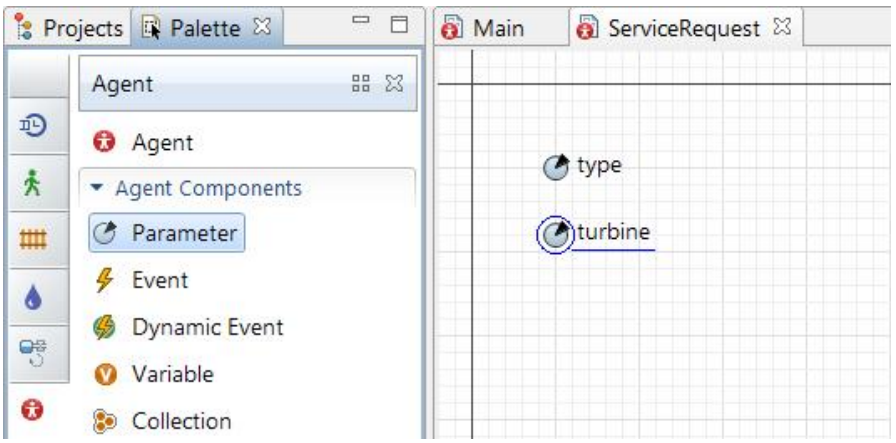
43. If you do not find it, right mouse click the *Wind Turbine* project in the **Projects** view. Select **New** > **Option List**.



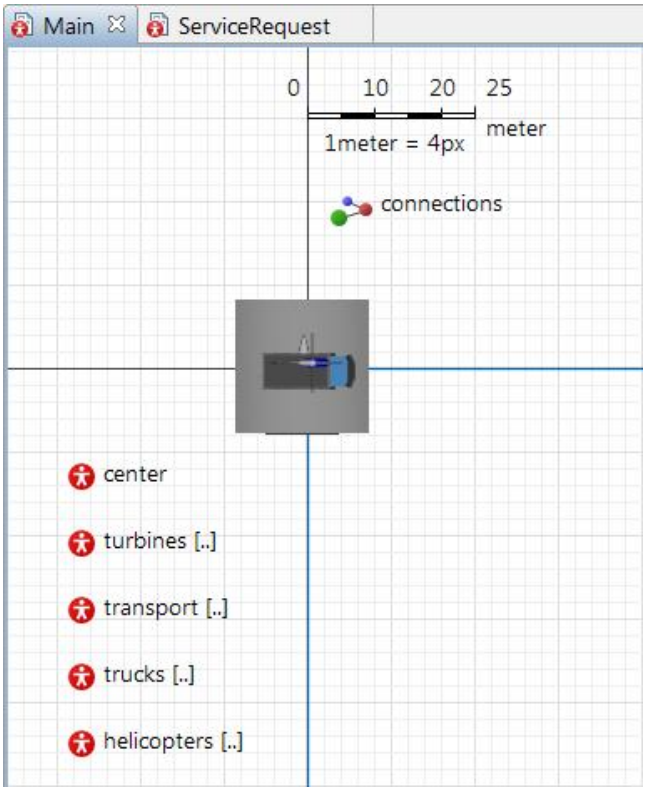
44. In the window that appears, type *TransportType* as the **Name** of the **Option List**, and *AUTO* and *AVIA* in the **Specify elements** section. Now you should find **Option Lists** in the model elements tree.



45. After you click **Finish**, AnyLogic will open the *ServiceRequest* agent type diagram. There you can find the parameter *type* that we have created. Add one more **Parameter** from the **Agent** palette. Call it *turbine* and select its **Type** from the drop-down list: **Turbine**. Now *ServiceRequest* is fully set up.



The agent elements will appear on *Main*. You can notice that the populations are marked with the **[..]**. Move the agents beyond the presentation window and place the center of each animation figure in the axis origin.



You can find the environment settings in the properties of *Main*, section **Space and network.**

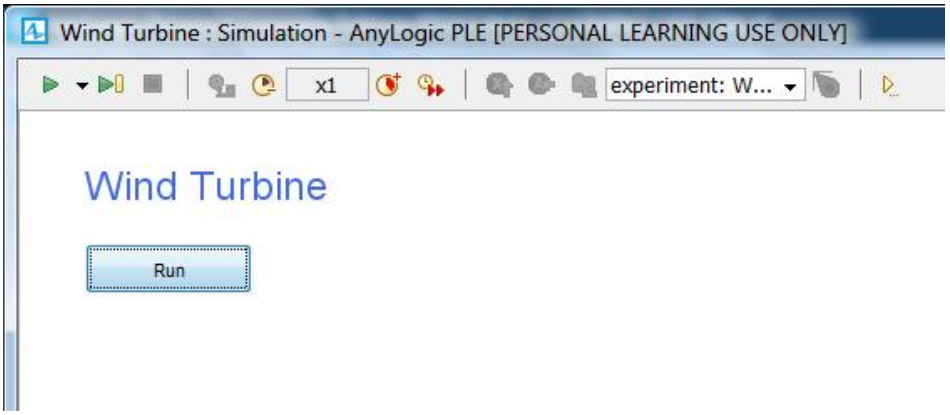
As the last step in this phase, change the scale of **Main** to match the rest of the agents.

46. Build your project by clicking the **Build Model** toolbar button. If there are some errors in your model, the building fails and the **Problems** view appears listing all the errors found in your model. Double-click an error in the list to open the location of the error and fix it. After the model is successfully built, you can start it. Running the simulation, you automatically bring the current model up to date.

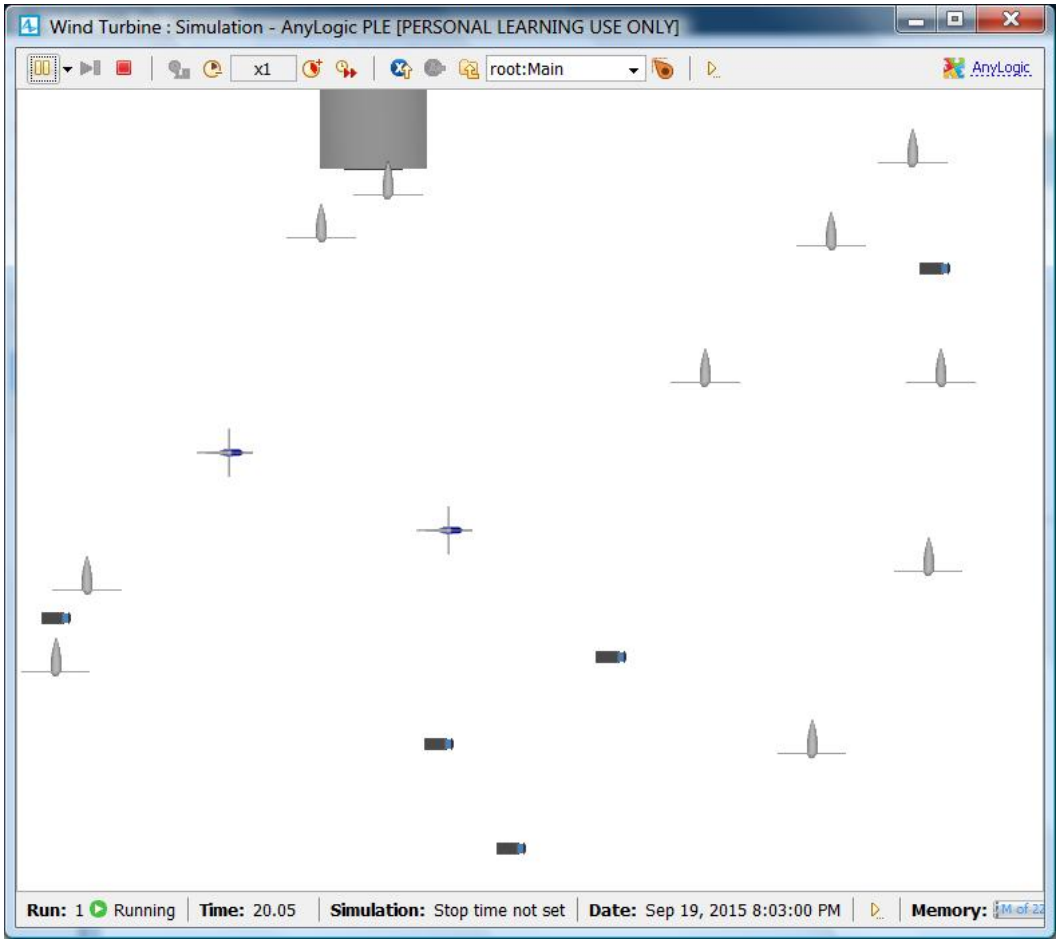
47. Choose the experiment you want to run from the drop-down list of the **Run** toolbar button. Your simulation experiment is called **Maintenance/Simulation**. Later on you can use the **Run** toolbar button to start the previously run experiment.

48. Having started the model, you will see the presentation window. It displays the presentation designed for your simulation experiment. AnyLogic automatically places the show-bench title and the button enabling running your model and switching to the presentation designed for the *Main* agent type.

49. Click **Run** button.



50. We have only started developing the model. At this moment we can see the maintenance center, turbines and our transport types distributed randomly in the continuous space.



Next, we need to place the transport - trucks and helicopters - in the maintenance center.

Phase 2. Setting up the transport base

We will be defining model logic and processes on each agent type diagram in the next phases of model development. We will start with *Main* configuration which sets trucks and helicopters position in the space.

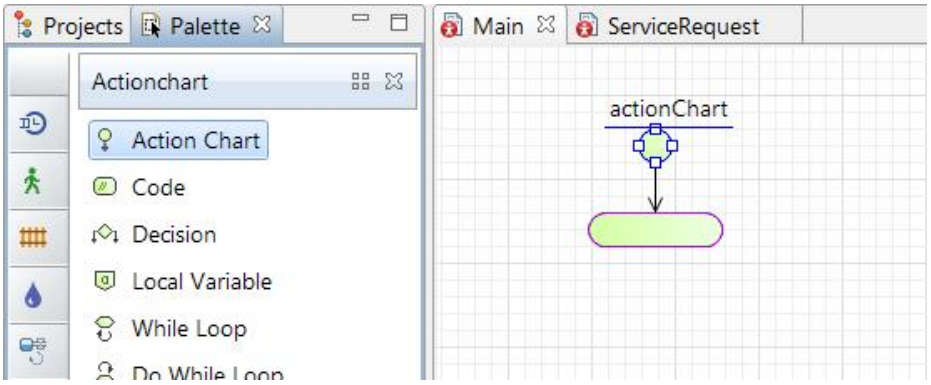
Complex simulation modeling cannot go without algorithms that usually perform some data processing or calculations. AnyLogic supports *actioncharts* - structured block charts allowing defining algorithms graphically in the style of structured programming. We use widely known extension of an approach suggested by Dijkstra that splits algorithms into sub-sections with a single point of entry. It states that three ways of combining programs—sequencing, selection, and iteration—are sufficient to express any computable algorithm. The style reduces understanding an algorithm to understanding each structure on its own.

Action charts are very helpful since using them you can define algorithms even if you are not familiar with syntax of Java operators.

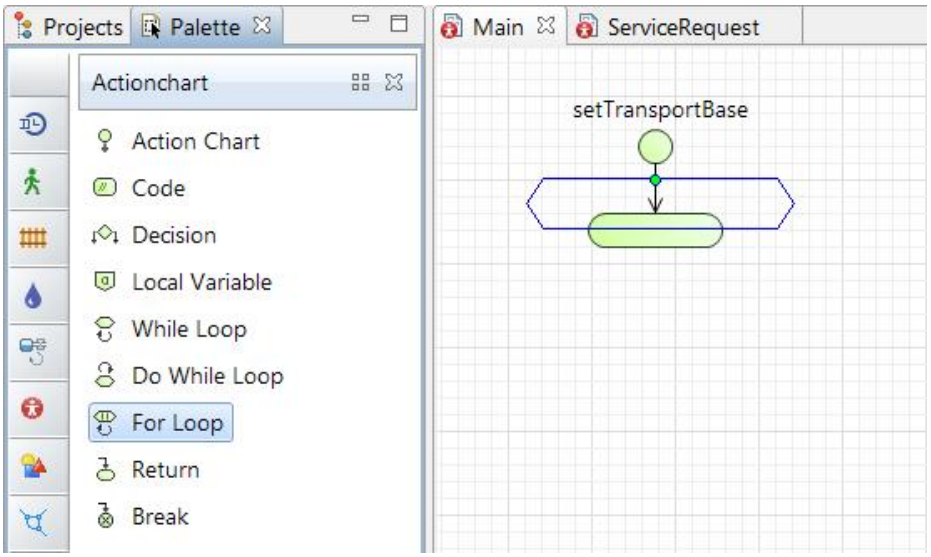
In fact, action chart visually defines a function and you can use ordinary AnyLogic functions as before. However, using *actioncharts* gives you one more evident benefit: it visualizes the implemented algorithm, making it more intuitive to other users.

1. Go to *Main* and open the **Actionchart** palette in the **Palette** view.

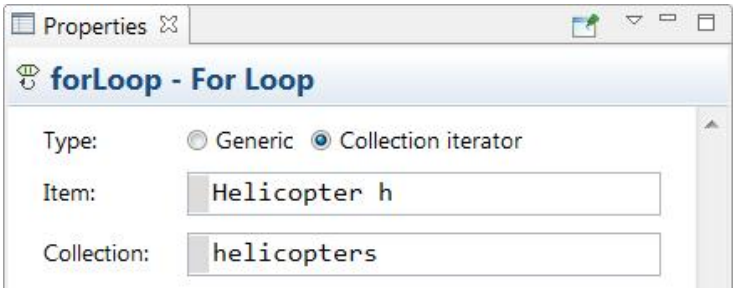
2. Drag the **Action chart** element into *Main*.



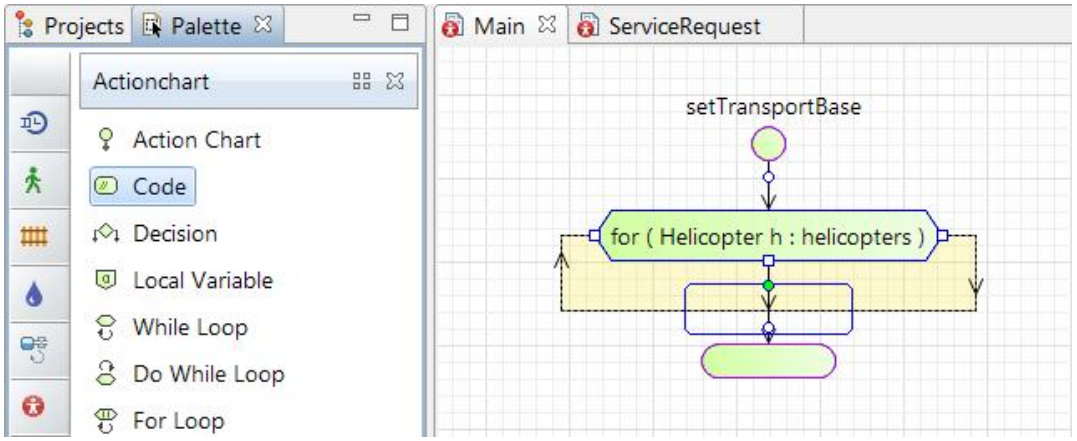
3. Call it *setTransportBase*. This actionchart does not return anything, it is **Just action**. Next, insert the element **For loop** in this actionchart. When working with actionchart elements, make sure you add them in the right place:



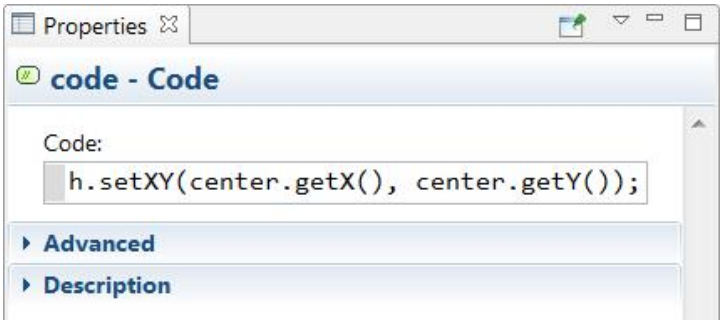
4. This for loop is of *Collection iterator* **Type** for *helicopters* **Collection**. Define **Item** as *Helicopter h*.



5. Next, insert **Code** into this for loop.

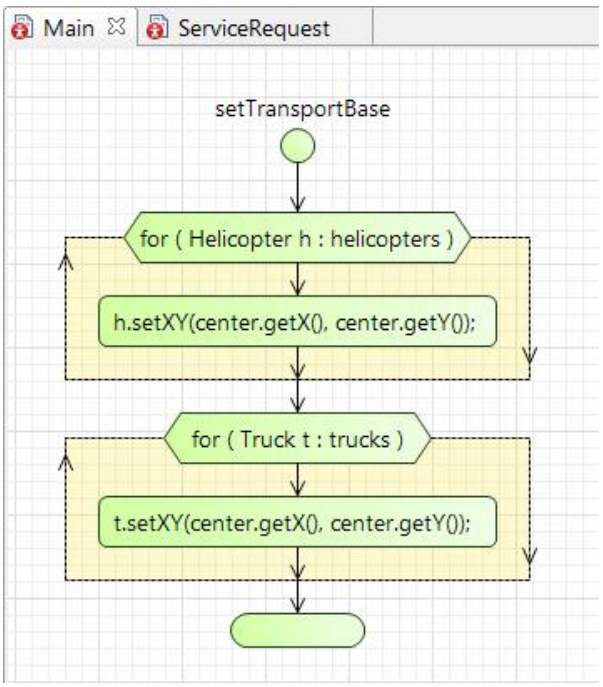


6. We want the helicopters to be initially located at the maintenance center coordinates: *h.setXY(center.getX(), center.getY());*



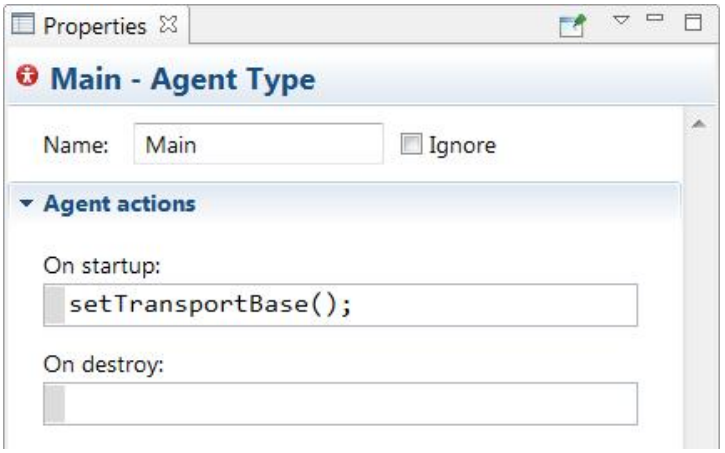
7. Add another block of **For loop** with the **Code** in this actionchart and configure it to place trucks at the maintenance center, same as helicopters.

8. You can resize the actionchart elements for the code text to fit in:

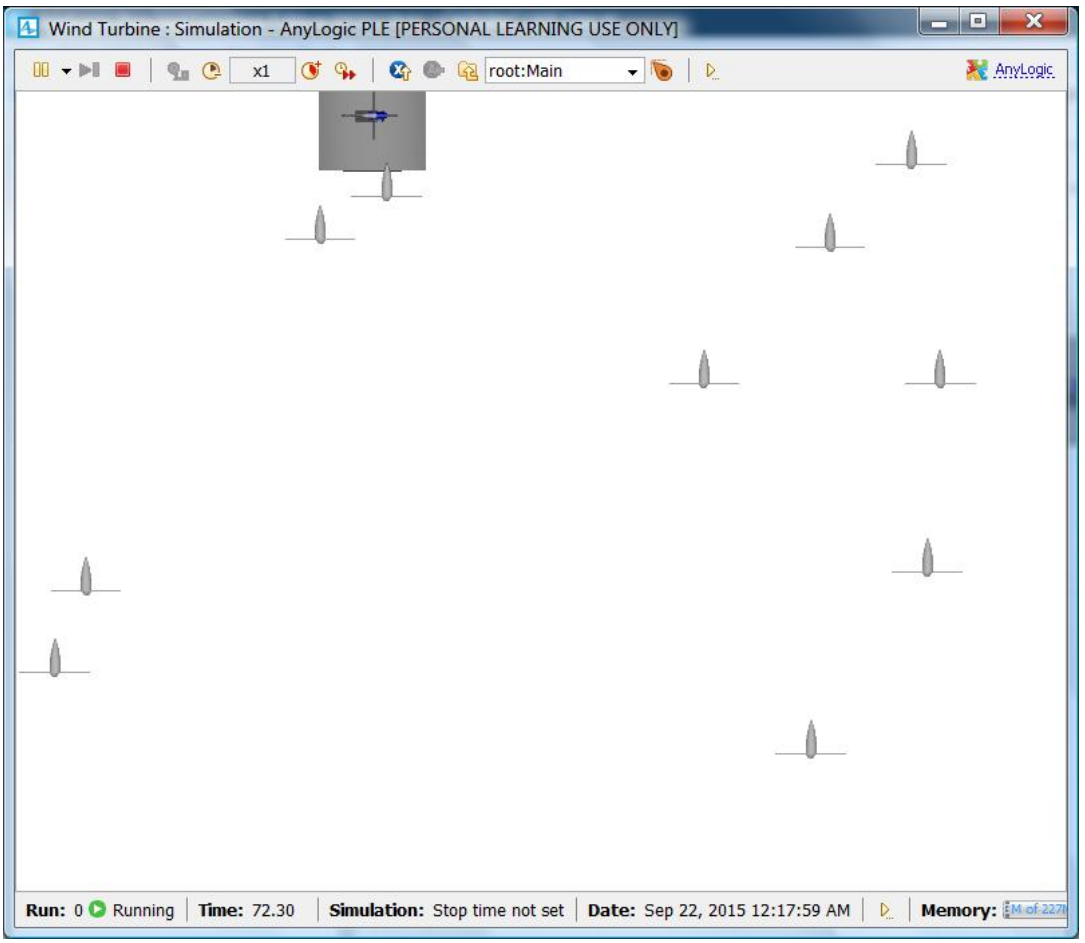


We have created the actionchart, called *setTransportBase*, that represents the function which places the transport at the maintenance center. This is where we need the transport types to be located initially at the model start.

Select *Main* in the model tree (or click anywhere in its graphical editor) and go to its properties. Expand the properties section **Agent actions** and tell it to run our function **On startup** of the model: *setTransportBase();*



Now you can run the model again. You will see that the trucks and helicopters are "sitting" now in the maintenance center. We will define their movement and all other agents behavior in the following phases of model development.



Phase 3. Defining Transport agent behavior

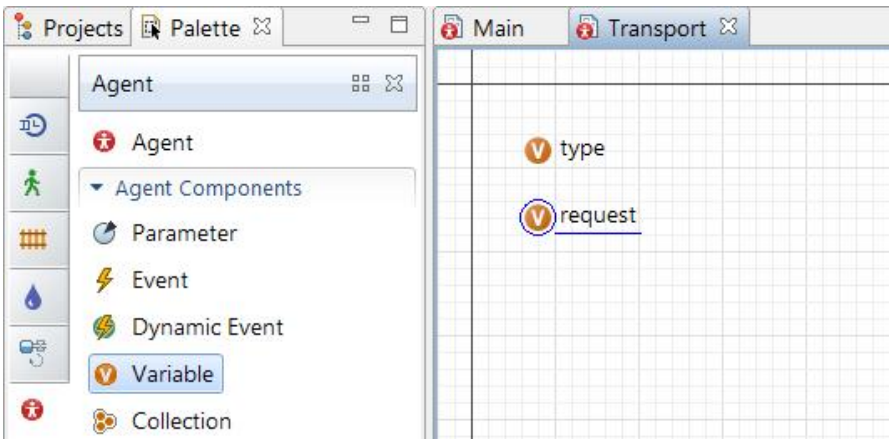
We will create the initial *Transport* agent type configuration.

Statechart is the most advanced construct to describe event- and time-driven behavior. For some objects, this event- and time-ordering of operations is so pervasive that you can best characterize the behavior of such objects in terms of a state transition diagram – a statechart.

Statechart has states and transitions. Transitions may be triggered by user-defined conditions (timeouts or rates, messages received by the statechart, and Boolean conditions). Transition execution may lead to a state change where a new set of transitions becomes active. States in the statechart may be hierarchical, i.e. contain other states and transitions.

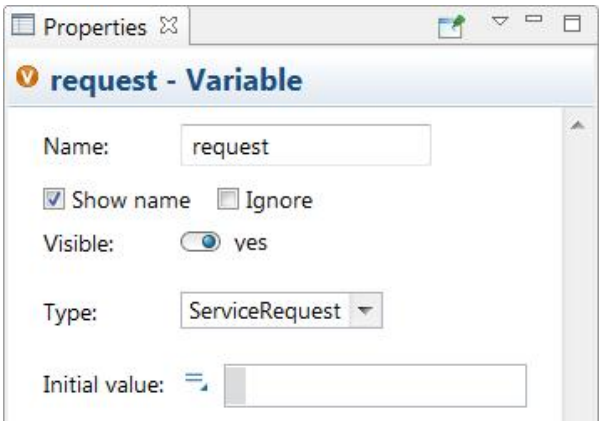
Statechart is used to show the state space of a given algorithm, the events that cause a transition from one state to another, and the actions that result from state change.

1. Go to the **Projects** view and double-click *Transport* to open it. Add two **Variable** elements from the **Agent** palette.

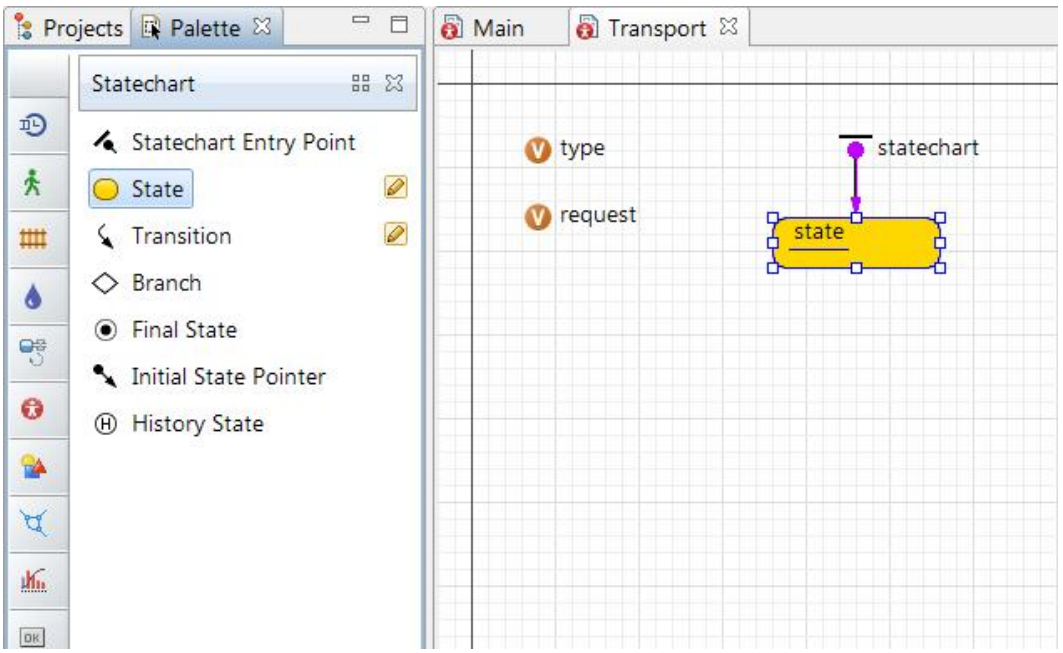


2. As you remember, we want to use both trucks and helicopters as maintenance transport in our model. We will use the variable *type* to differentiate between them. The variable *type* should be of **TransportType** type so we can refer to the options *AUTO* and *AVIA*.

3. The variable *request* should be of type **ServiceRequest**:

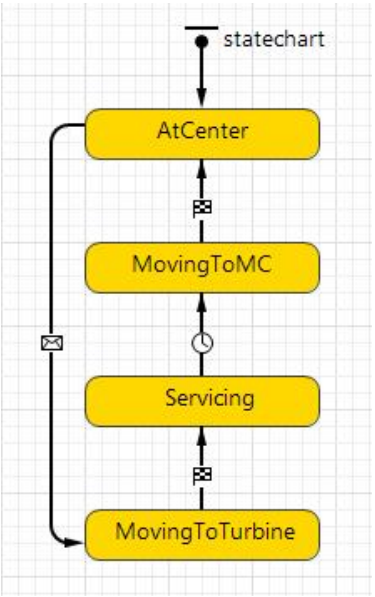


4. We will use the **Statechart** palette elements to define the transport agent type behavior. Drag the **Statechart Entry Point** element into the graphical editor of *Transport*. It defines the beginning of the whole diagram. Add a **State** by dragging it from the **Statechart** palette and dropping the state under the statechart entry point. To add a transition, double-click **Transition** in the palette. Its icon will turn into , it means the tool is in the *drawing mode*. Now you can draw transitions by clicking in the graphical editor. Since the transitions must be connected to the states, you can click in the state when drawing to ensure proper connection. When the elements are successfully connected, AnyLogic displays a cyan-colored point in the place of connection.



5. At this phase we will only create an "empty" *statechart* that consists of four states and four transitions of three different types.

6. The states are called *AtCenter*, *MovingToMC*, *Servicing*, *MovingToTurbine*. These are basically all actions and routes of trucks and helicopters in the model. Make sure you follow the structure that is shown in the figure below.



7. The "internals" of some of these states and transitions refer to the model elements we have not yet defined. On the other hand, we need this statechart now because some of those model elements that we are going to define in further phases refer to this statechart and states in it. That is why we will finish configuring *Transport* in the very end of model development process.

   Let us next define the transport type choice according to request type and the availability of trucks and helicopters at the maintenance center.

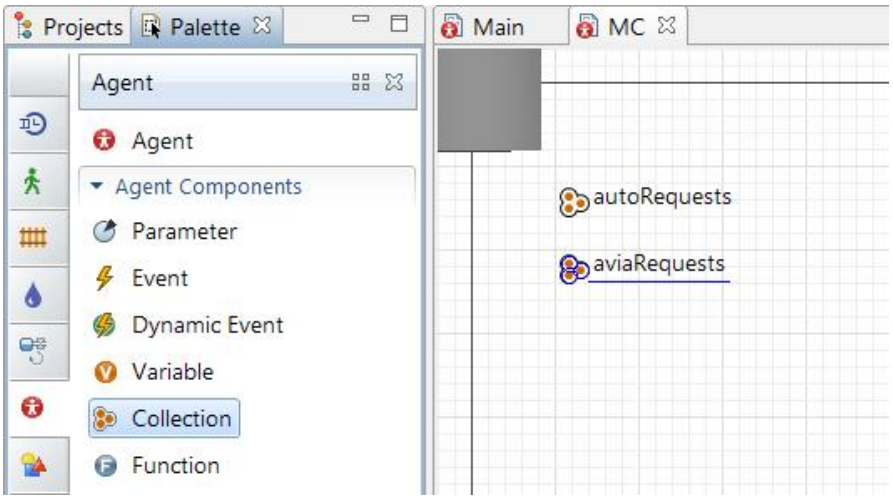
Phase 4. Defining Maintenance Center behavior

We will create an actionchart where the maintenance center decides what transport is needed, a truck or a helicopter, and checks if they are available to perform maintenance.

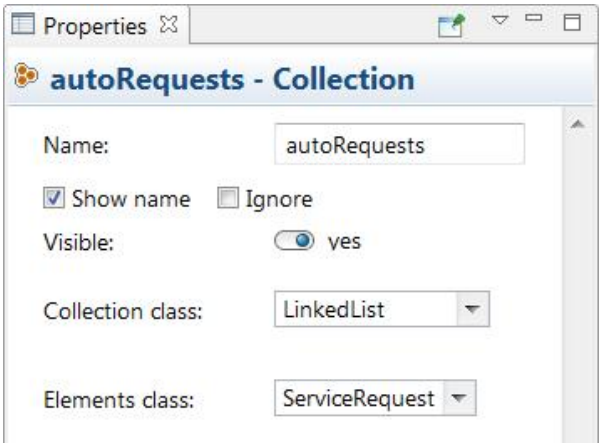
Agent can contain variables. Variables are generally used to store the results of model simulation, to model some data or object characteristics, changing over time. AnyLogic supports two types of variables – variables and collections.

A collection represents a group of objects, known as its elements. They are used for defining data objects that group multiple elements into a single unit. Collections are used to store, retrieve and manipulate aggregate data. Typically, they represent data items that form a natural group, such as a queue (in this case elements represent people waiting in a queue), or an autopark (elements are trucks), or a telephone directory (a collection performs mapping of names to phone numbers).

1. Open *MC* diagram. Add two **Collection** elements from the **Agent** palette.

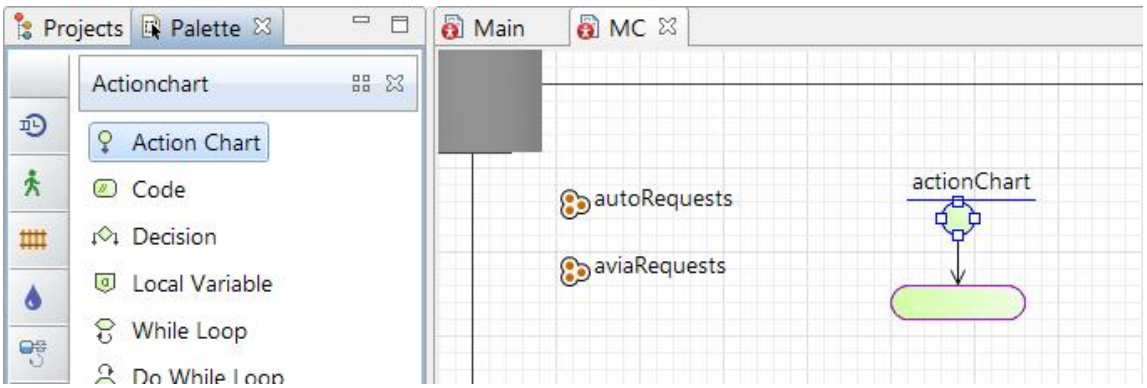


2. The collections called *aviaRequests* and *autoRequests* are both of *LinkedList* **Collection class** and *ServiceRequest* **Element class**.

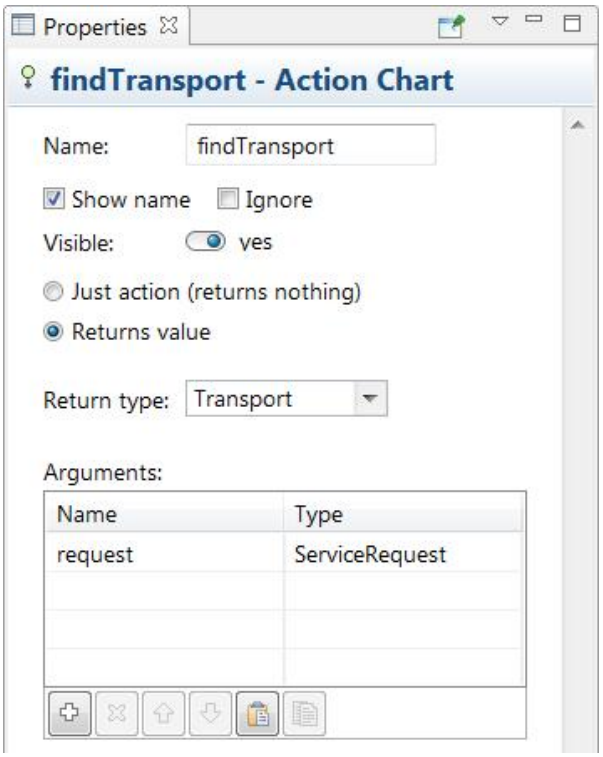


3. Open *MC* diagram. We will add the process diagram here using **Actionchart** palette elements.

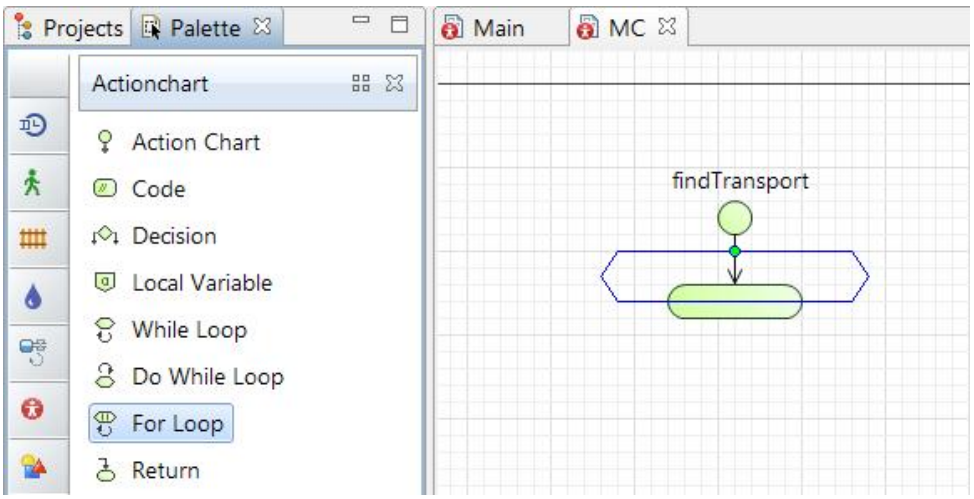
4. First drag the **Action Chart** element from the palette into the graphical editor.



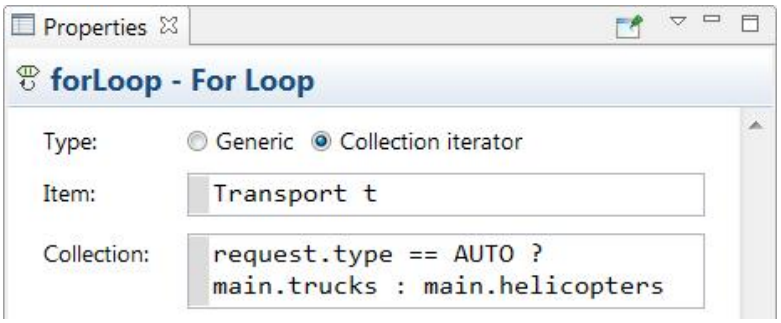
5. Call it *findTransport*. This actionchart **Returns value** of type **Transport**. Its arguments should be specified in the table below: *request* argument of **Type** *ServiceRequest*:



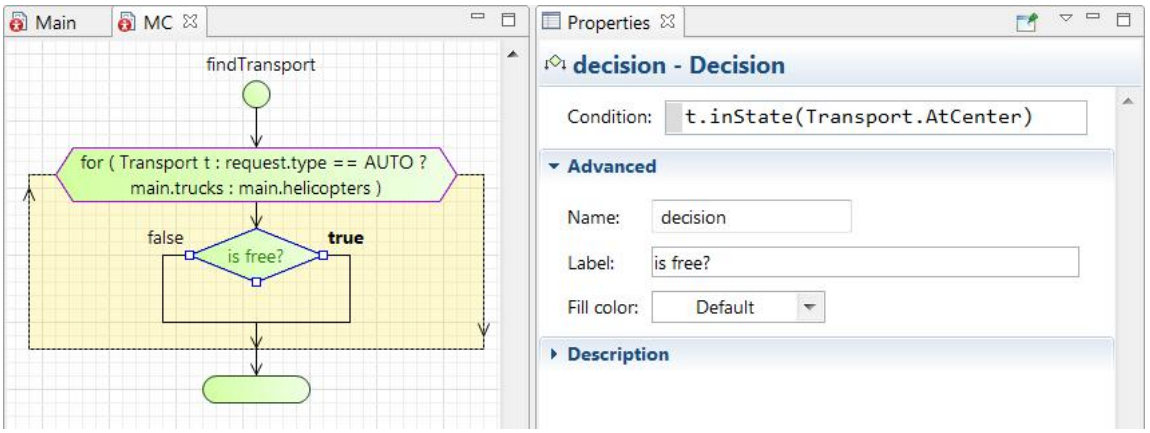
6. Next, insert the element **For loop** in our *findTransport* actionchart. When working with actionchart elements, make sure you add them in the right place:



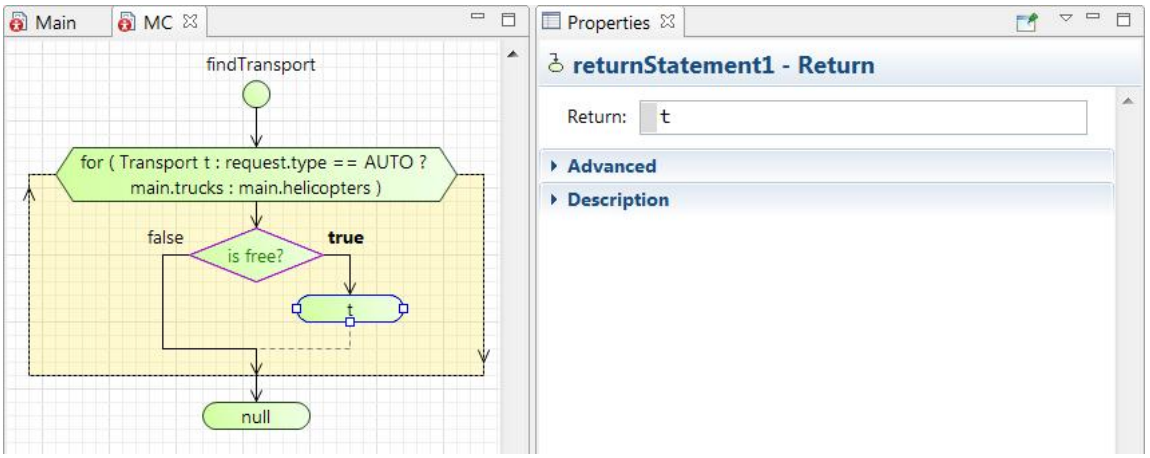
7. The type of this for loop is **Collection iterator**. Specify **Item** (*Transport t*) and **Collection** (*request.type == AUTO ? main.trucks : main.helicopters*) as shown in the figure below - it depends on the request if a truck or a helicopter is needed. When finished, you can resize the element for the text to fit in the form.



8. We need to add **Decision** that will check if the transport is available. The condition (*t.inState(Transport.AtCenter)*) says that the transport item should be in the *AtCenter* state from the statechart we have created on *Transport*. Note that now the element displays the **Label** text in the diagram:



9. Insert **Return** element in the *true* branch. Tell it to return *t*. As the last step select the actionchart return statement for *false* and specify **Return**: *null*.



10. This actionchart will check if there is an available truck or helicopter at the maintenance center.

In AnyLogic you enter pieces of code in numerous properties of various model elements. It is important to understand exactly where you are writing the code (which agent type and method it belongs to) and how you can access other model elements from there.

The elements of the same agent type are accessed simply by their names.

To access an element of an embedded object you should put a dot "." after the embedded object name and then write the element name. For example, we referred to the function *getX()* of the maintenance *center* when creating the actionchart on *Main*: *center.getX()*. If the embedded object is replicated, you should say which one exactly you want to access.

To access the object that shares the container with the current one, first get up to the container object and then get down to another agent.

To access the container of the current object you should call *<its name>*. Note that there is a special element, **Link to upper level agent**, on every agent type diagram whose agents live in that agent type (here it is *Main*). The link is called after this upper level agent type and allows us to access it easily by simply referring to its name. For instance, to get to the element **trucks[..]**, which is contained in *Main*, from within *MC*, we call *main.trucks* in *MC*.

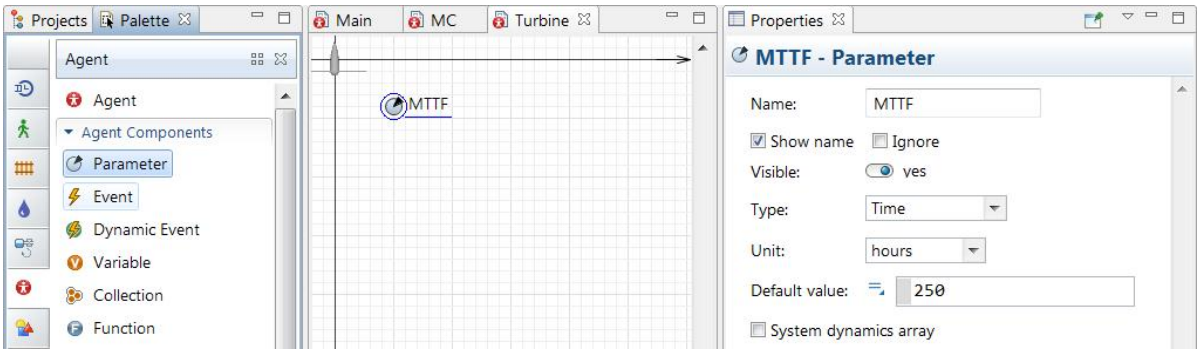
Next let us configure *Turbine*.

Phase 5. Defining Turbine behavior

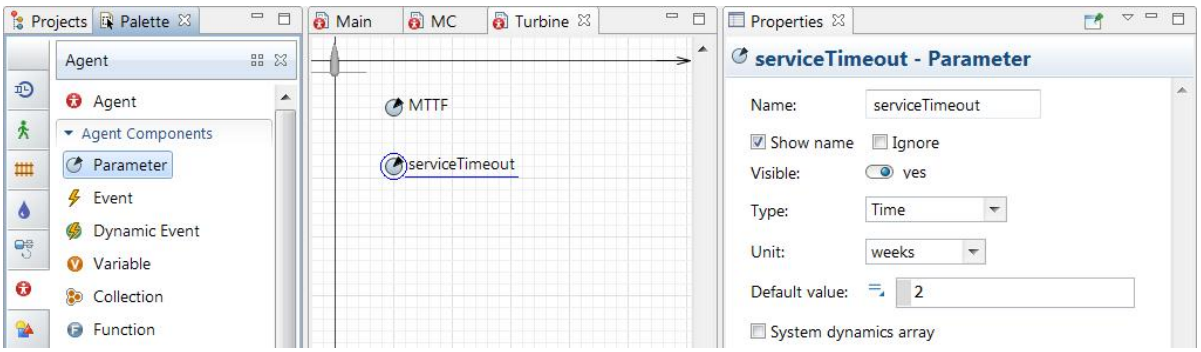
In this phase we will define the behavior of turbines with a statechart: when and how they get out of service or get scheduled maintenance. Also, we will set up the turbine animation: we want the blades to stop rotating when a turbine is out of service; moreover, every turbine animation figure has color indicators of its turbine state.

1. Double-click *Turbine* in the model tree to open its diagram. Start with adding two **Parameter** elements from the **Agent** palette into the graphical editor.

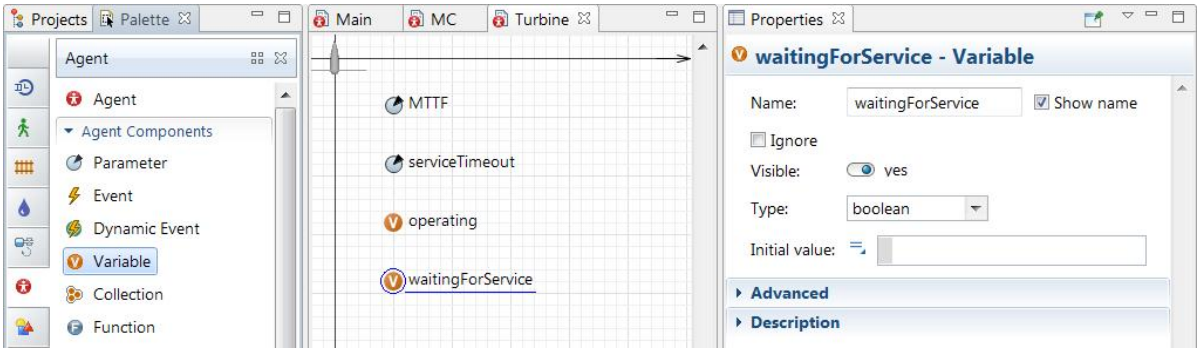
2. The parameter called *MTTF* (mean time to failure) is of type **Time** and its **Default value** is *250 hours* (select hours in the **Units** property).



3. The second parameter, *serviceTimeout*, is also of type **Time** and its **Default value** is *2 weeks*.



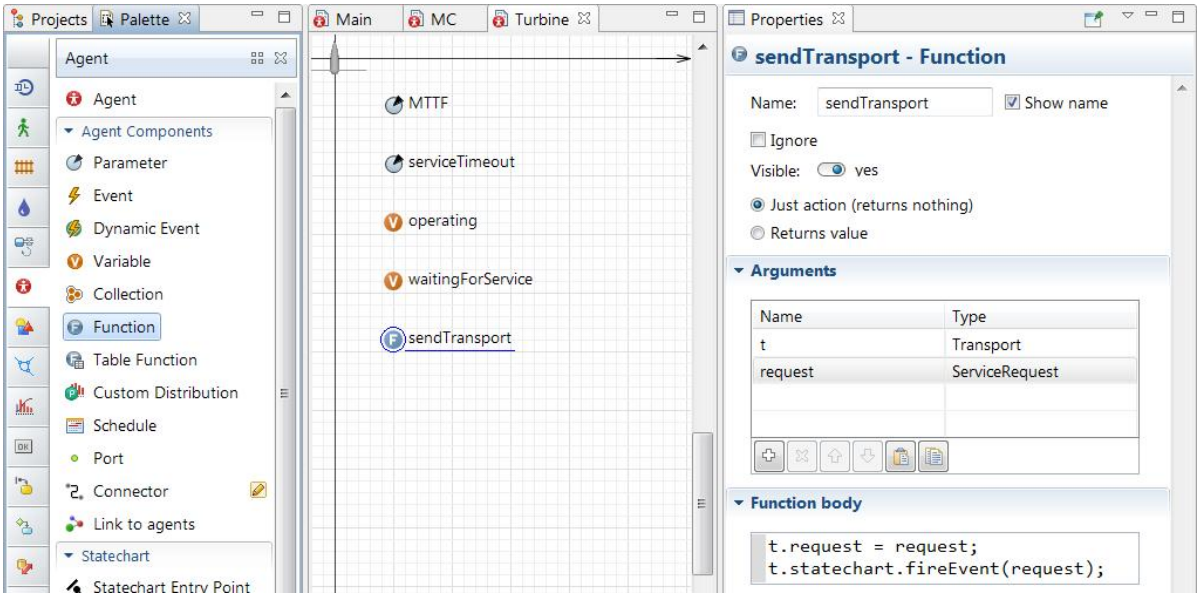
4. Next we need to add two **Variables** that we will use later in the statechart diagram to define states. The variables *operating* and *waitingForService* are both of type **boolean**. The variable *operating* has **Initial value**: *true* because all turbines initially appear working.



5. Add a **Function** from the **Agent** palette called *sendTransport*. This function is **Just action** and it has two arguments that you can set up in the **Arguments** section: *Transport t* and *ServiceRequest request*. This function sends transport to a turbine when needed using the statechart on *Transport*.

   t.request = request;

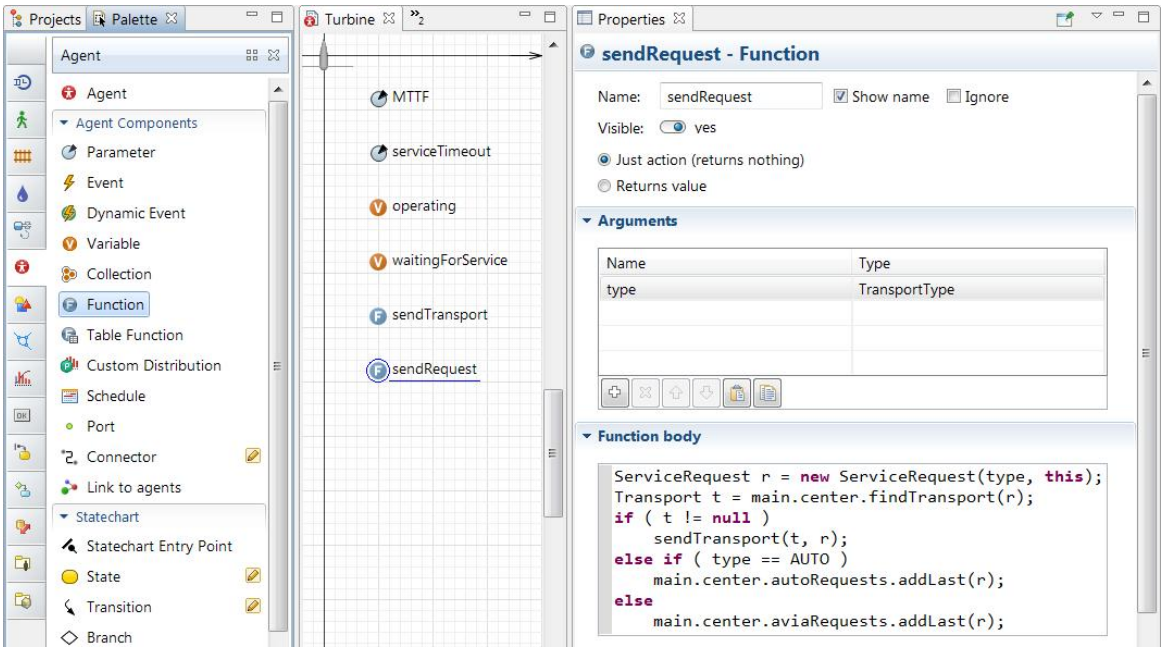
   t.statechart.fireEvent(request);



6. Add another **Function**. Call it *sendRequest* and specify its properties as shown in the figure below. In the properties section **Arguments**, add *TransportType type* argument.

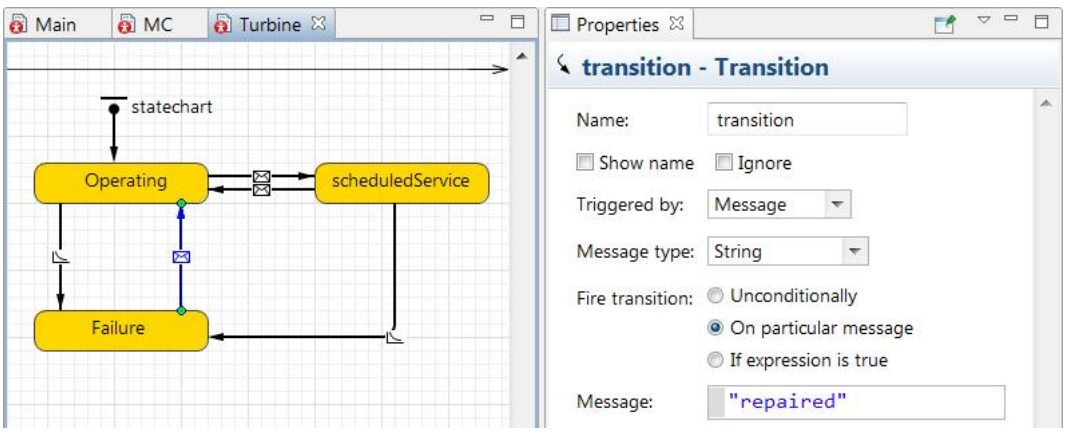
The function body refers to the actionchart *findTransport* that we have created on *MC*. When a turbine sends a service request, the maintenance center should send transport of a specified type, *AUTO* or *AVIA*.

```
ServiceRequest r = new ServiceRequest(type, this);
Transport t = main.center.findTransport(r);
if ( t != null )
   sendTransport(t, r);
else if ( type == AUTO )
   main.center.autoRequests.addLast(r);
else
   main.center.aviaRequests.addLast(r);
```



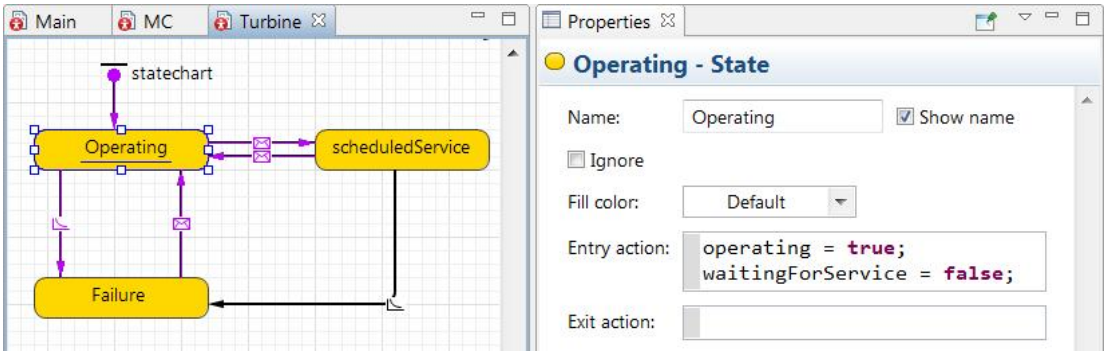
7. Now we are ready to create a statechart. Open the **Statechart** palette and drag and drop into *Turbine* diagram the **Statechart Entry Point** element.

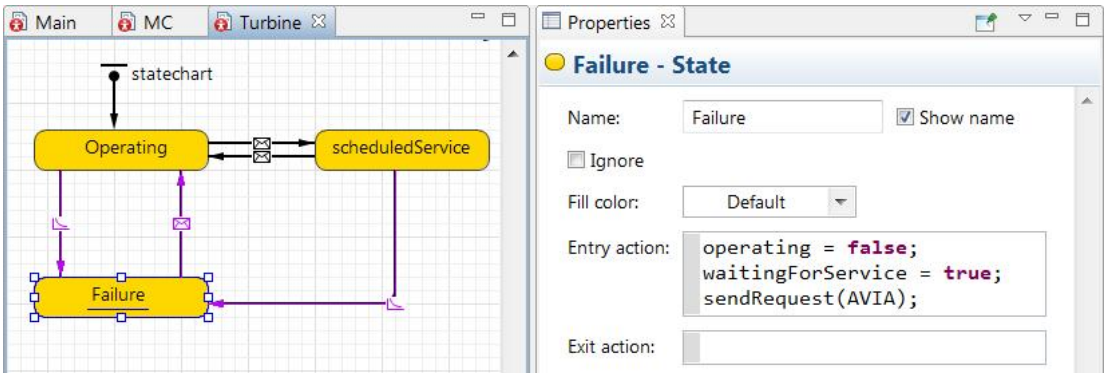
8. Add three states and several transitions between them as shown in the figure below. The transition that goes back from *Failure* to *Operating* state is triggered by particular Message *"repaired"*.

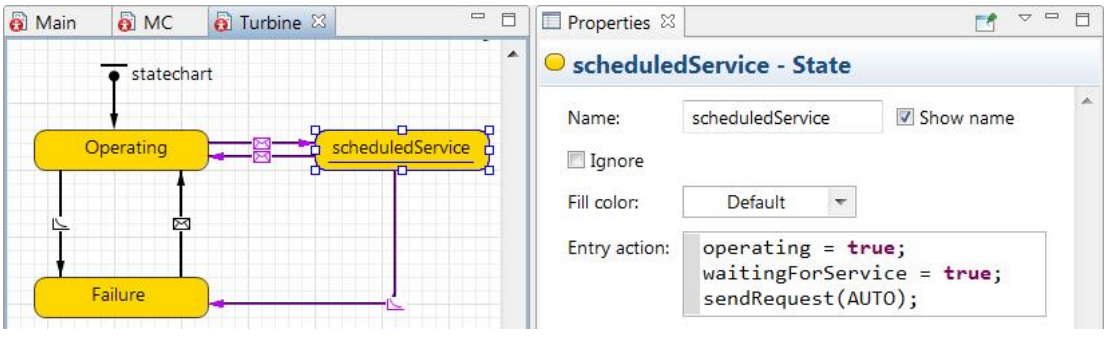


9. Each turbine state has its own **Entry action**. The state of a turbine depends on the variables *operating* and *waitingForServices*. When a turbine needs repair, it is in the *Failure* state sending a maintenance request for *AVIA* transport (a helicopter). When it is time for the scheduled service, a turbine sends a request for *AUTO* type of transport - a truck.

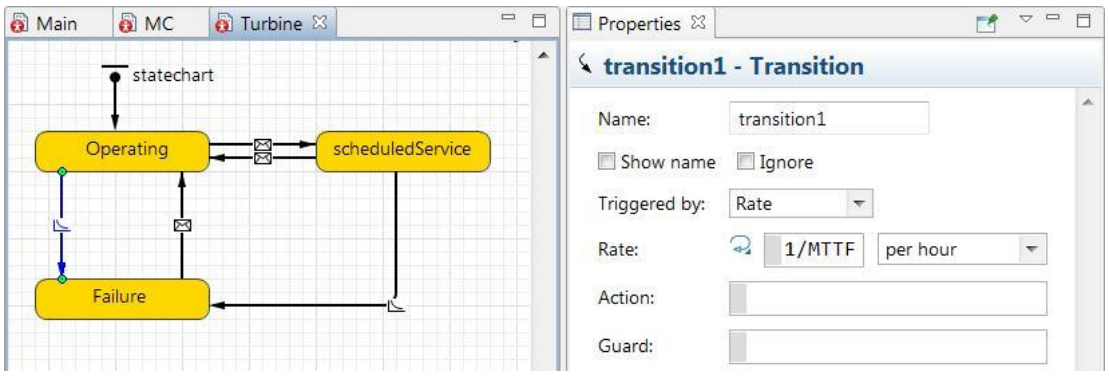
State	Entry action
Operating	*operating = true;* *waitingForService = false;*
Failure	*operating = false;* *waitingForService = true;* *sendRequest(AVIA);*
scheduledService	*operating = true;* *waitingForService = true;* *sendRequest(AUTO);*



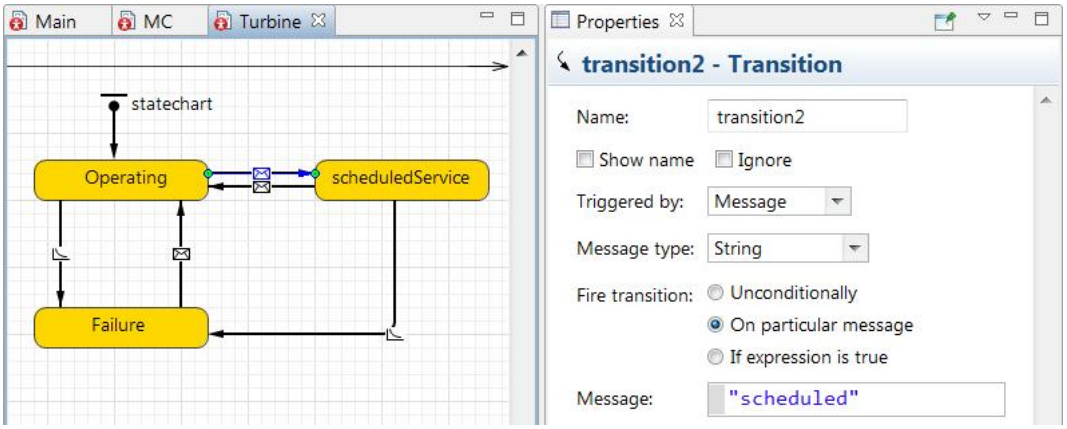




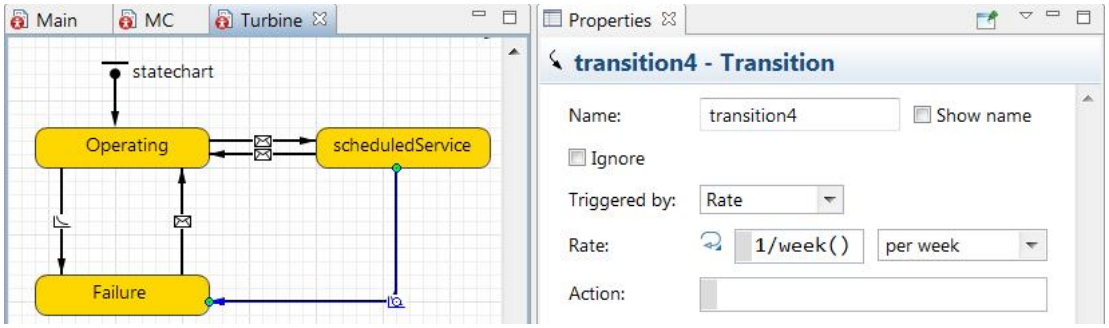
10. The transition from *Operating* state to *Failure* is triggered by **Rate** that equals *1/MTTF* - once in the mean time to failure period.



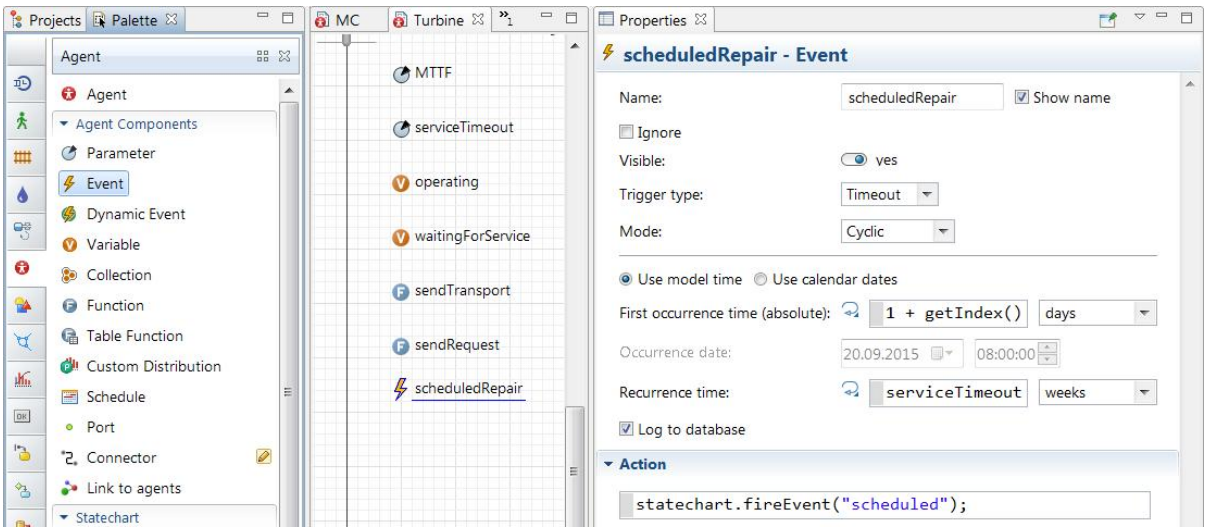
11. When an operating turbine receives a message *"scheduled"*, it means, a turbine needs scheduled service and the corresponding state is triggered. When a turbine receives a message *"repaired"*, the service is finished, and it can get back to the *Operating* state.



12. The last transition we need to put until the statechart diagram is complete - from *scheduledService* to *Failure*. This transition is triggered by **Rate**, *1/week()*.

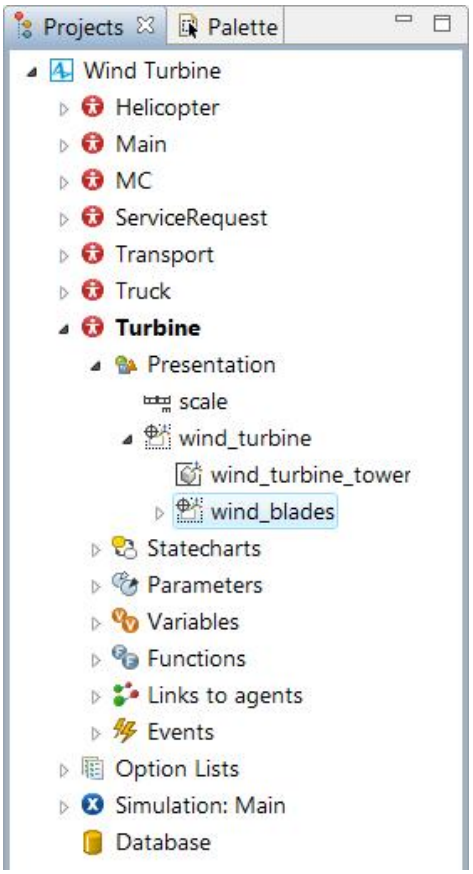


13. Add a cyclic timeout **Event** called *scheduledRepair*. This event will happen when a turbine needs a scheduled maintenance (according to *serviceTimeout*) that triggers the transition from *Operating* state to *scheduledService*: *statechart.fireEvent("scheduled");*

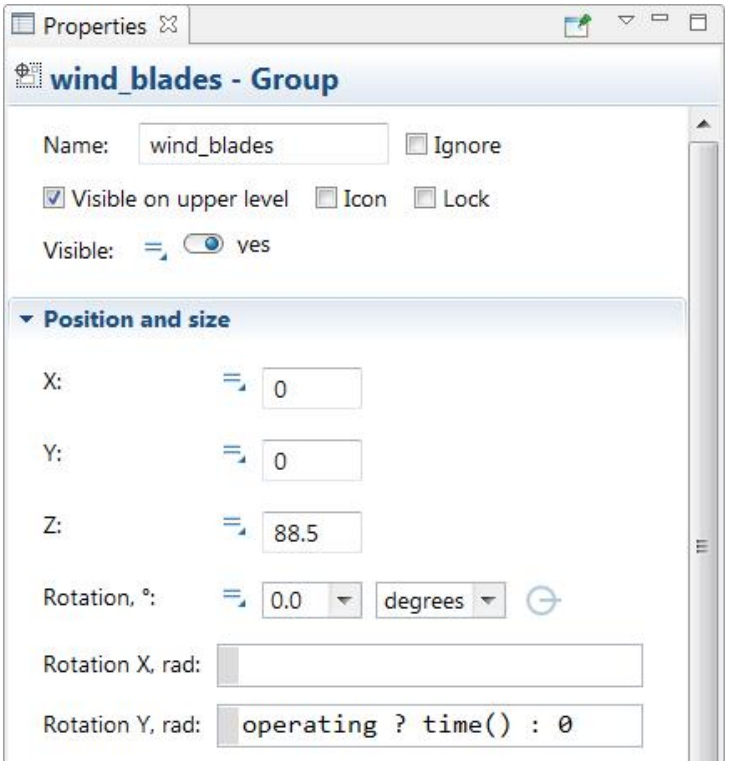


14. We need to change the properties of the turbine blades. The *Wind turbine rotating* figure is actually a group consisting of 2 3D figures: turbine tower and turbine blades. We need to open the properties of the blades.

15. You can select the blades with a mouse click in the graphical editor, or open the **Projects** view and keep expanding the model tree levels until you get to the **wind_blades** group:



16. Go to its properties. Open the section **Position and size** and type *operating?time():0* in the property **Rotation Y, rad**. Now the blades will be rotating when the turbine is working, and when it is out of service and needs repair the blades will remain still.



17. Open the **Presentation** palette and double-click the **Oval** element to enable the drawing mode.

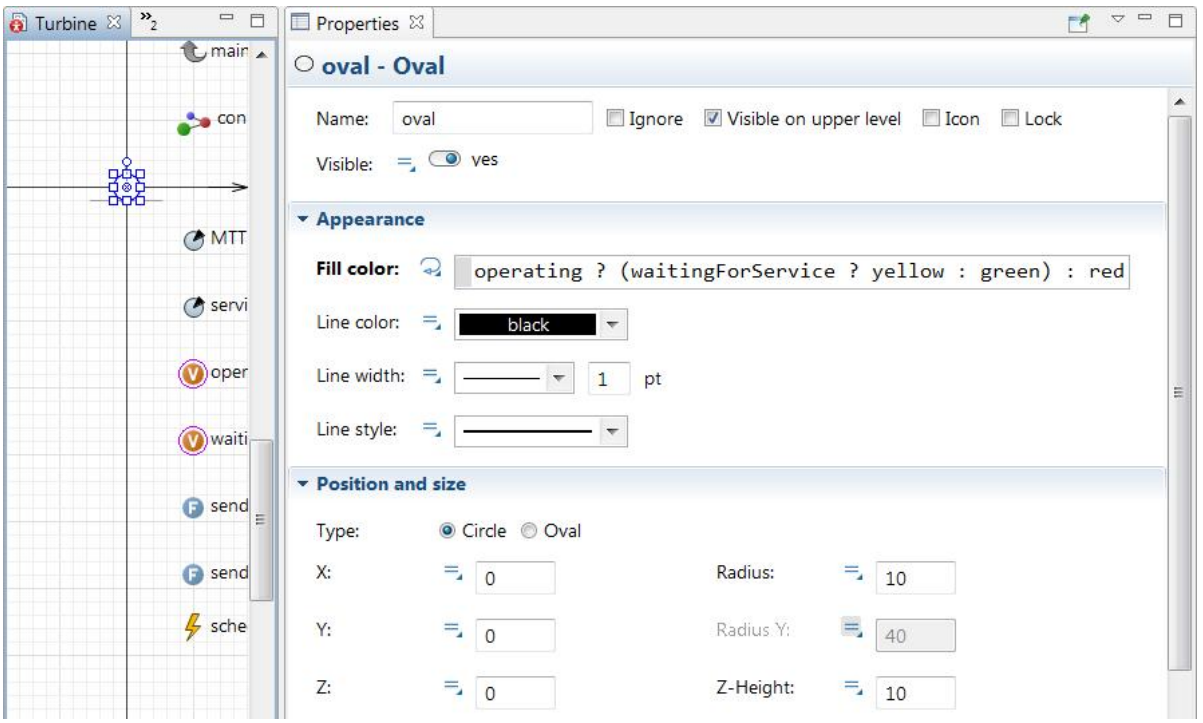
18. Draw a circle around the turbine animation figure with the radius around *10*.

19. Right-click the circle and select **Order** > **Send to Back** from the context menu.

20. Go to the **Appearance** section of the circle properties. Enter the expression that will be evaluated during the run-time for the **Fill color** to make it change according to the turbine state.

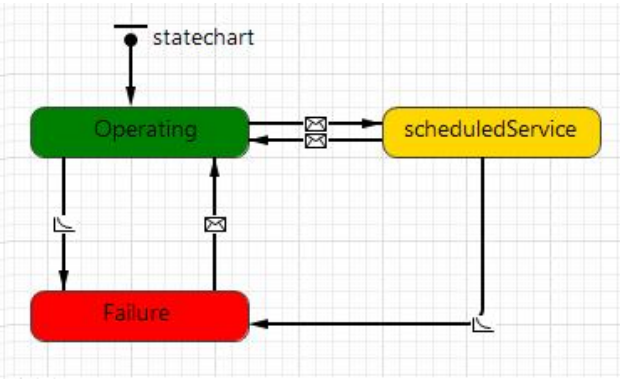
21. In order to switch to *dynamic value* mode for the property **Fill color**, click its icon , and then type:

    operating ? (waitingForService ? yellow : green) : red

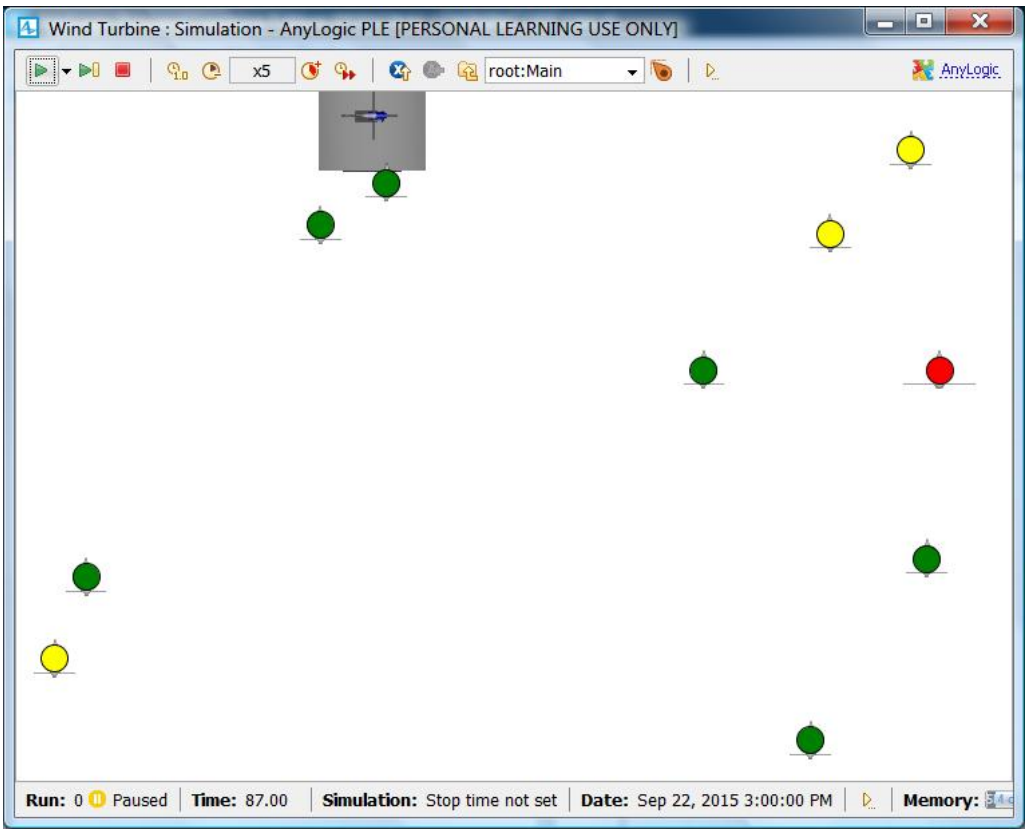


22. If a turbine is waiting for service the shape displays the yellow color, otherwise the circle is green which is the indicator for a working turbine, and if it is out of service the indicator displays the red color.

    Additionally, you can resize the statechart elements and set some custom colors for the states.



Run the model now. You will see that some turbines are working, some are waiting for the scheduled maintenance and some turbines need urgent repair.



Now we are ready to define the transport movement.

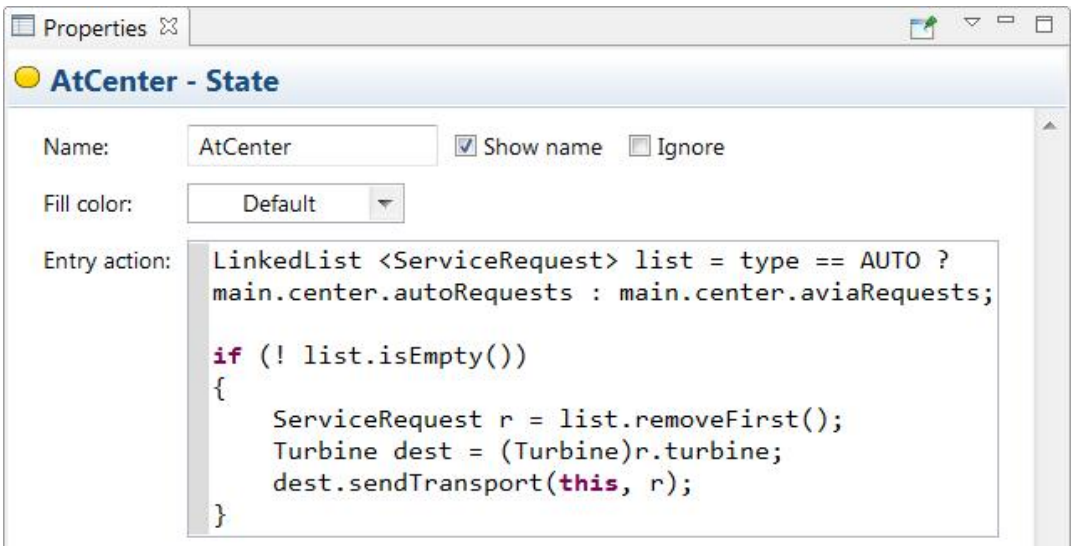
Phase 6. Finishing the Transport logic

In this phase we will add the final details of *Transport* statechart elements. We will finish defining the transport movement between the maintenance center and turbines.

1. Double-click *Transport* in the model tree to open its diagram. There you can find the statechart we have added before. Now we can configure its settings.

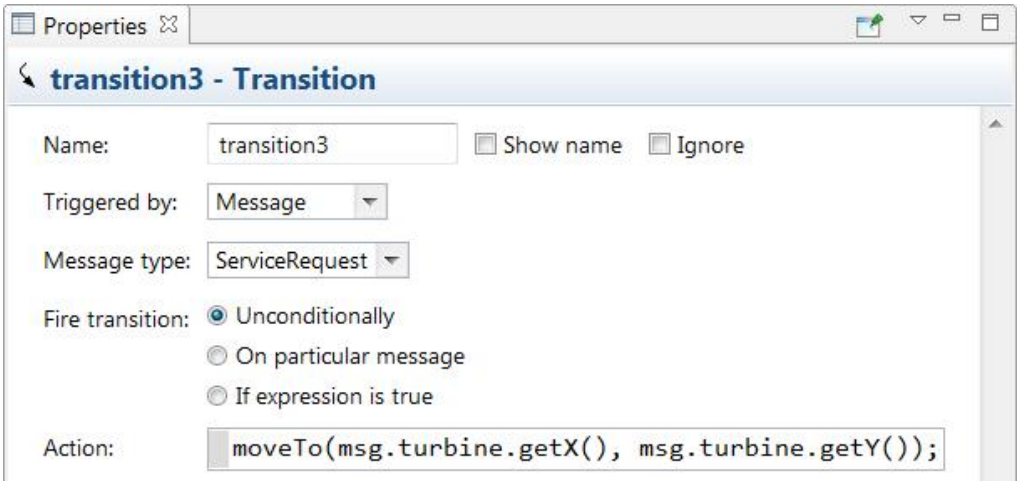
2. Select the state *AtCenter*. We need to specify its **Entry action**:

```
LinkedList <ServiceRequest> list = type == AUTO ?
main.center.autoRequests : main.center.aviaRequests;
if (! list.isEmpty())
{
  ServiceRequest r = list.removeFirst();
  Turbine dest = (Turbine)r.turbine;
  dest.sendTransport(this, r);
}
```

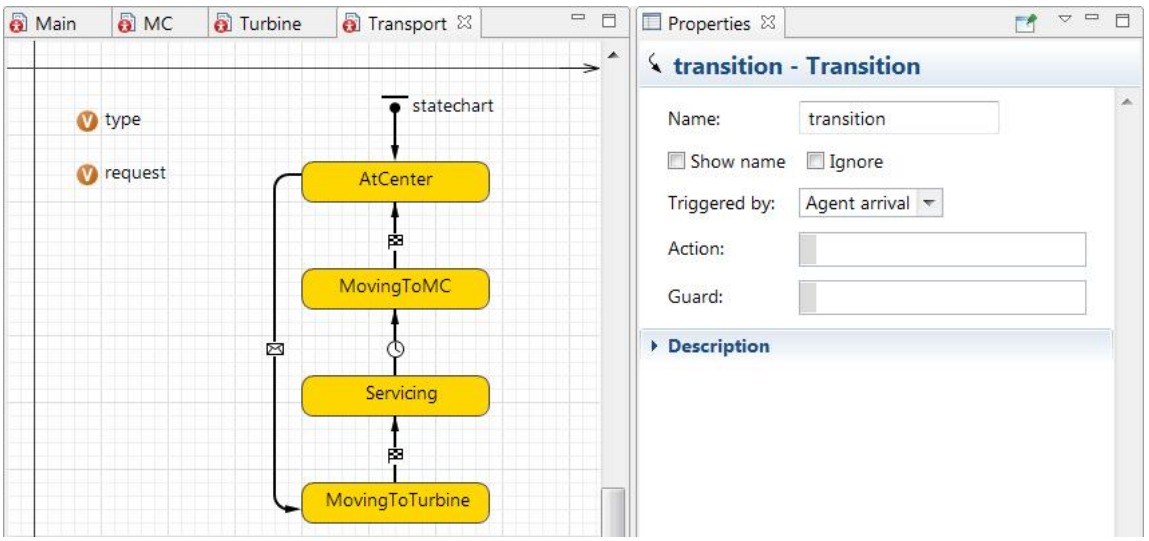


3. Three other states (*MovingToMC*, *Servicing*, *MovingToTurbine*) do not have any specific properties. We will define the logic of transport movement using different kinds of transitions.

4. The transition from *AtCenter* to *MovingToTurbine* is triggered by **Message** of type **ServiceRequest**. Since the transition is fired **Unconditionally**, you need to specify **Action** that the transport should take: move to the turbine coordinates: *moveTo(msg.turbine.getX(), msg.turbine.getY());*



5. The transition from *MovingToTurbine* to *Servicing* is triggered by **Agent arrival**. A lorry or helicopter that reached the broken turbine will start their tasks immediately after arrival. When the repair is finished and so the turbine signals, the transport can go home.

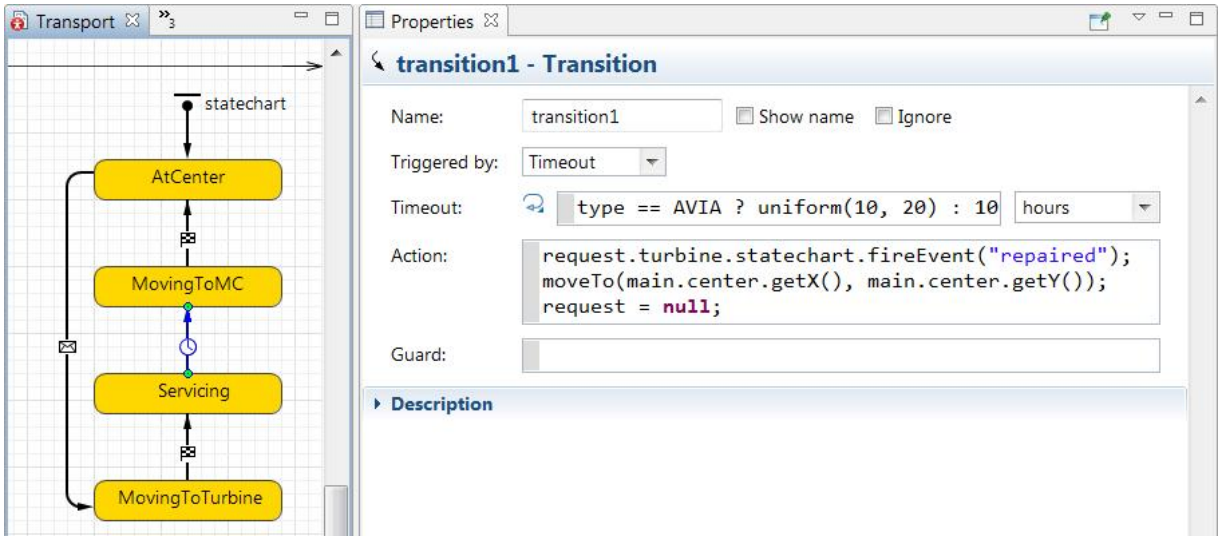


6. The next transition goes from *Servicing* to *MovingToMC*. The action to go back to the maintenance center is triggered by **Timeout** (*type == AVIA ? uniform(10, 20) : 10*) and the **Action** is

   request.turbine.statechart.fireEvent("repaired");

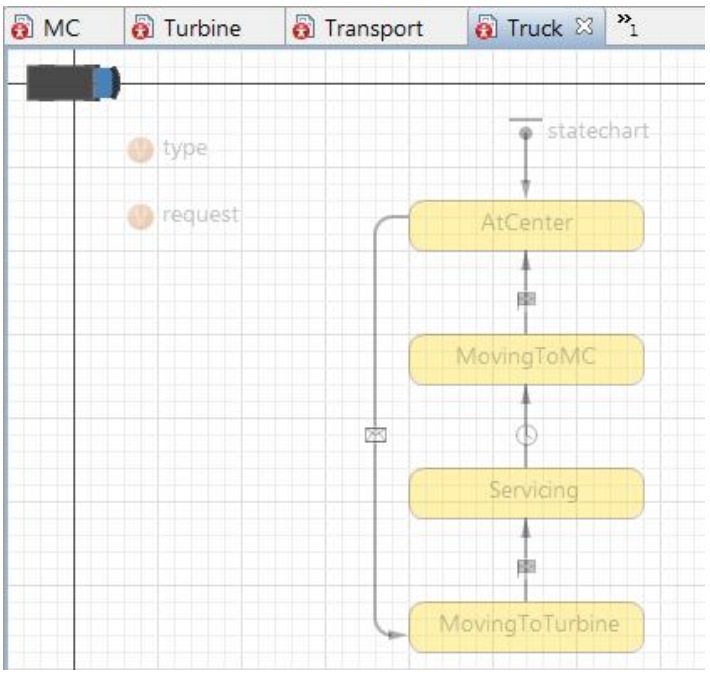
   moveTo(main.center.getX(), main.center.getY());

   request = null;

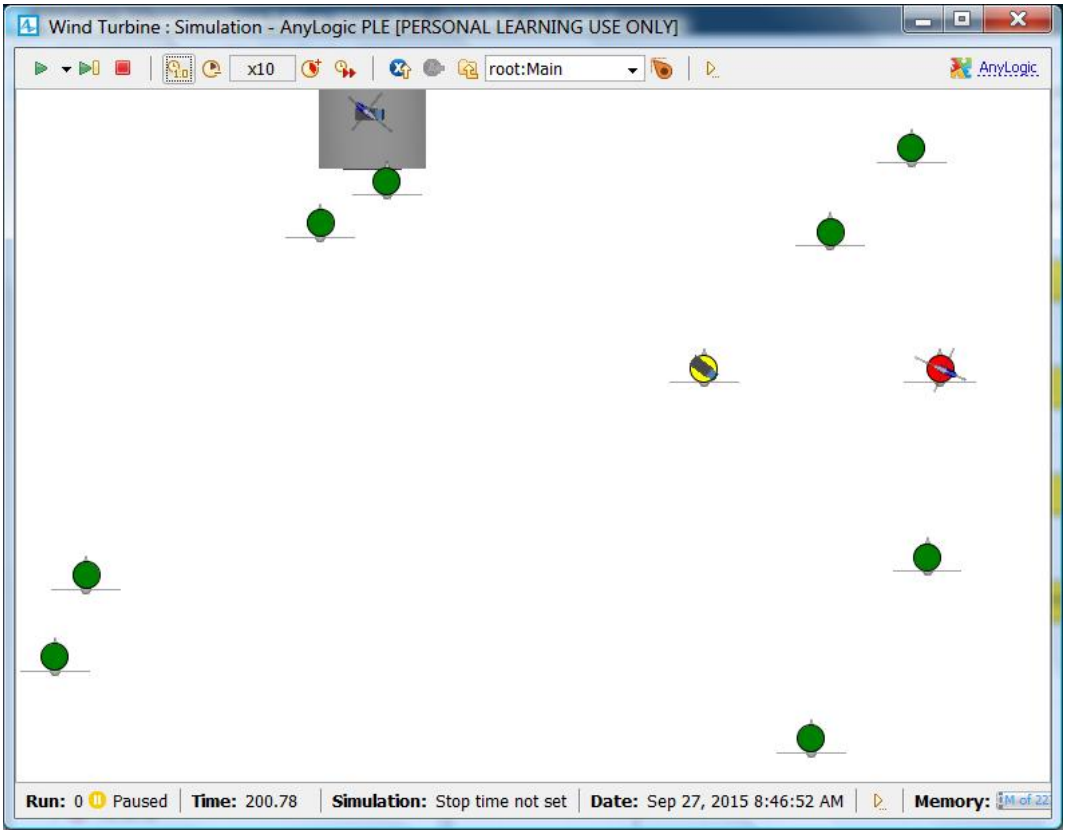


7. In the figure above you see that the last transition, from *MovingToMC* to *AtCenter*, is triggered by **Agent arrival**. When this happens, the transport is back at the maintenance center and remains available until another turbine needs service.

   If you open now *Truck* or *Helicopter* you will see that there is *Transport* elements projection. These elements are visible here and you can select them to see their properties for your convenience, but they are not editable here:



Run the model. Initially all turbines are operating (indicated with green). Then we can see how a truck moves to the turbine that needs scheduled maintenance (yellow turbines) and the helicopters are moving to the turbines which are out of service (red turbines) to perform urgent repair works.

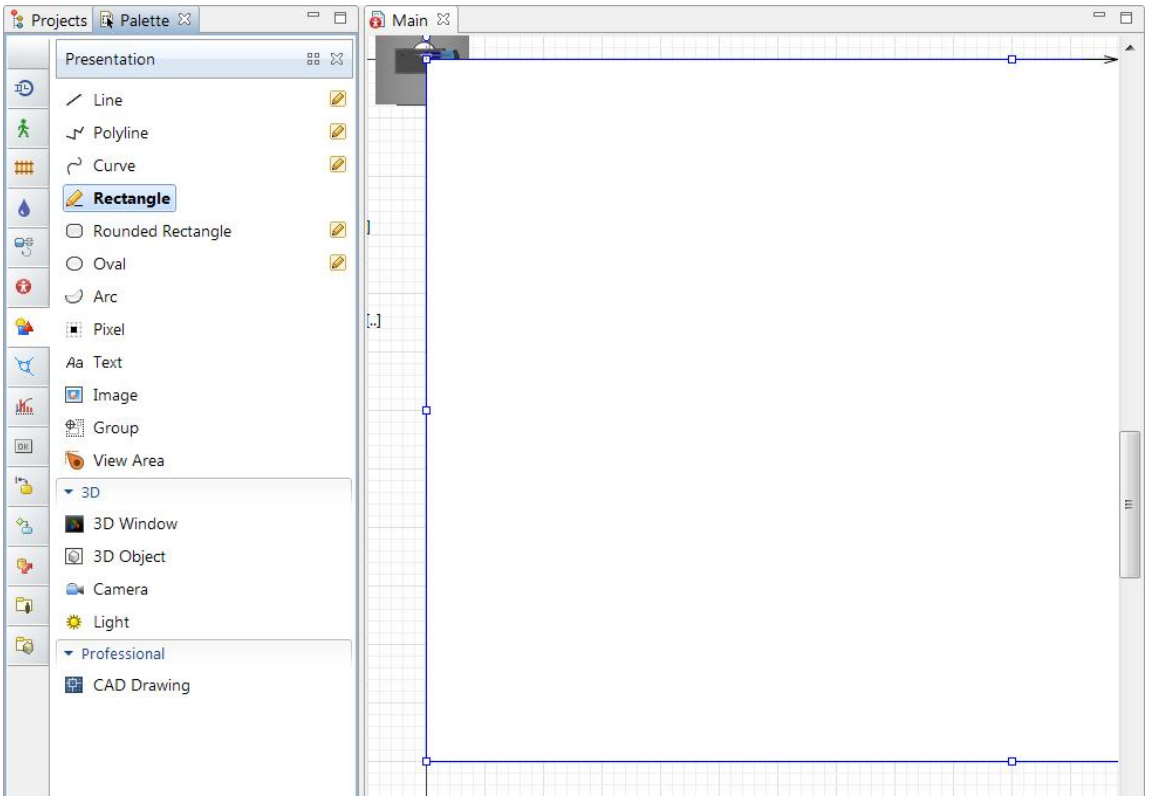


We have finished defining logic of the model, let us now add some animation elements, configure 3D animation view and explore the model during run-time.

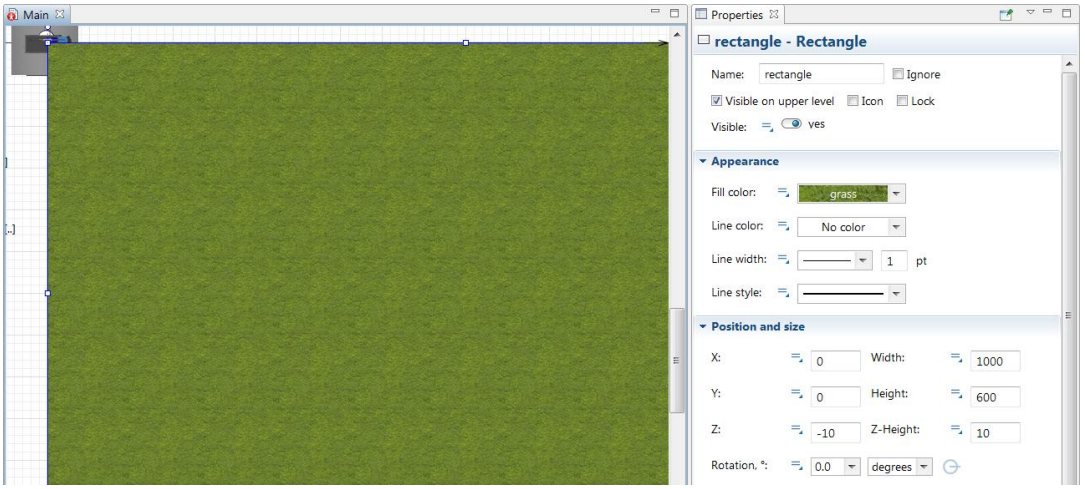
Phase 7. Running and exploring the model

This is the final phase of the model development.

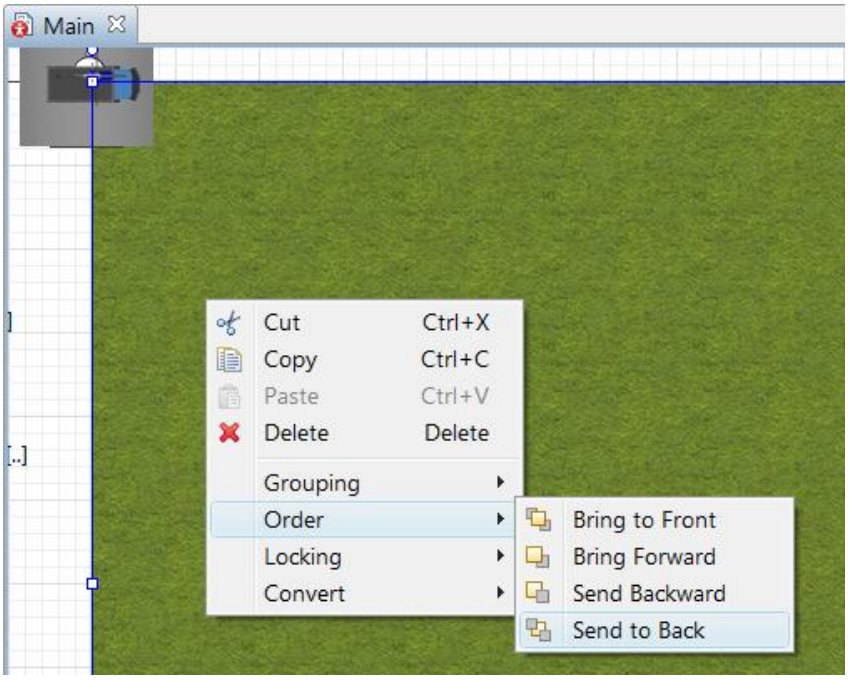
1. You can add some background for the agents animation with the help of the presentation shape **Rectangle**. Double-click **Rectangle** in the **Presentation** palette to enable the drawing mode. Then, click in the graphical editor of *Main* and drag the rectangle without releasing the mouse button until it takes necessary form.



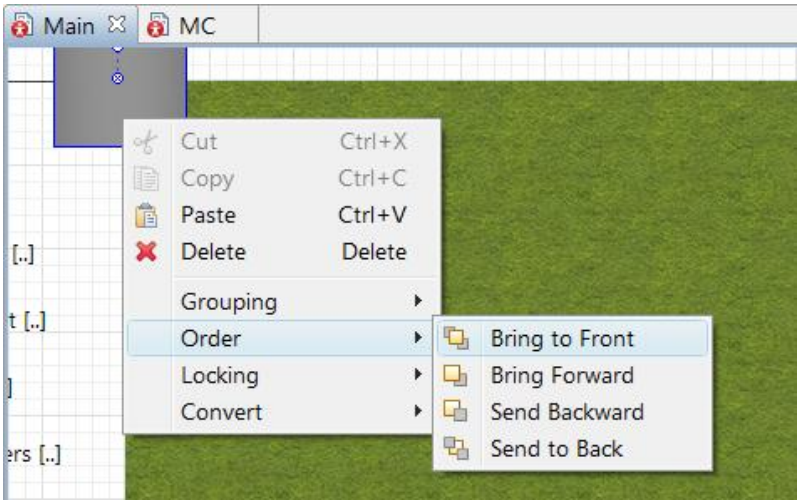
2. Go to the properties section **Appearance** and select the texture *Grass* as **Fill color** and *No color* for **Line**. Mind the **Z** and **Z-Height** properties. For instance, if **Z-Height** is set to *10*, you can set **Z** to *-10*, otherwise other animation figures will "drown" in this background because the surface they are moving on is 0 level.



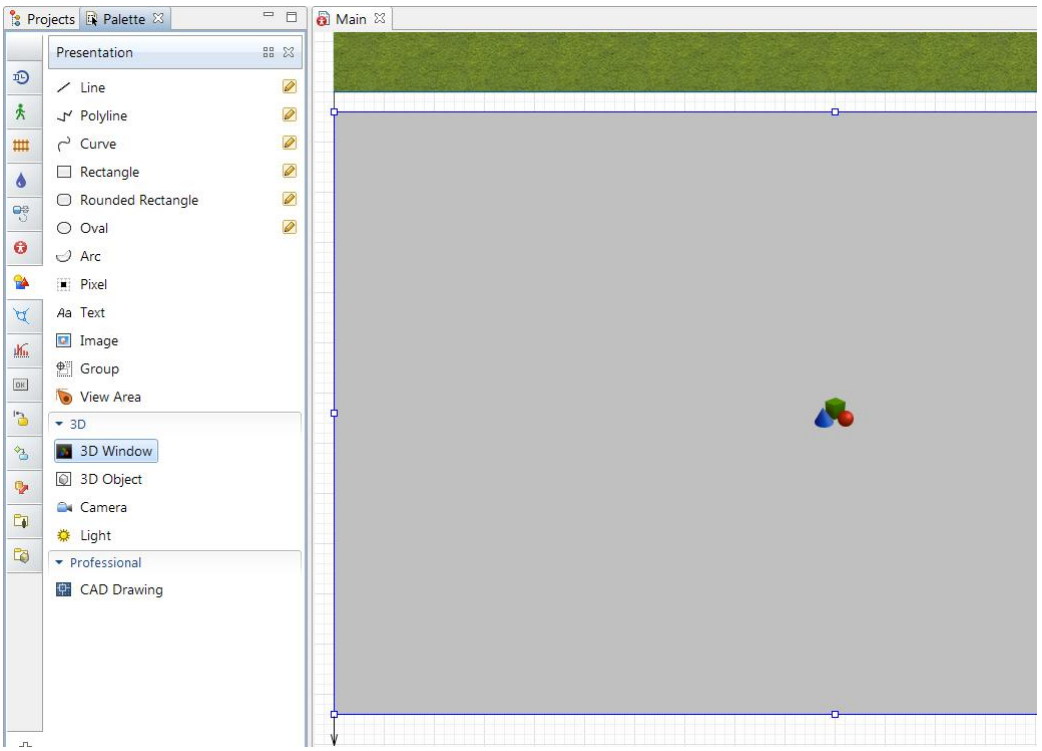
3. Now, since the rectangle is the latest shape added on this graphical editor, it is displayed over our agents's animation figures. Right-click the rectangle *area* and select **Order** > **Send to back**.



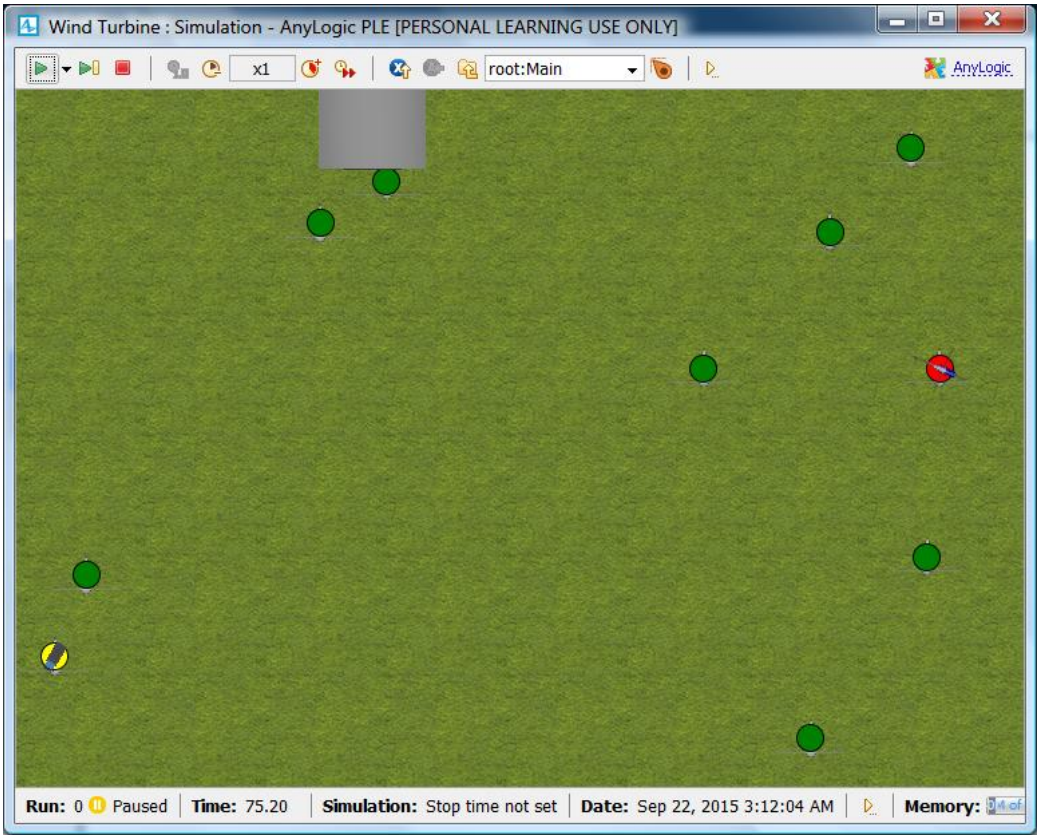
4. You can also right-click the *MC* animation figure hangar here and select **Order** > **Bring to front** to "hide" the transport behind it.



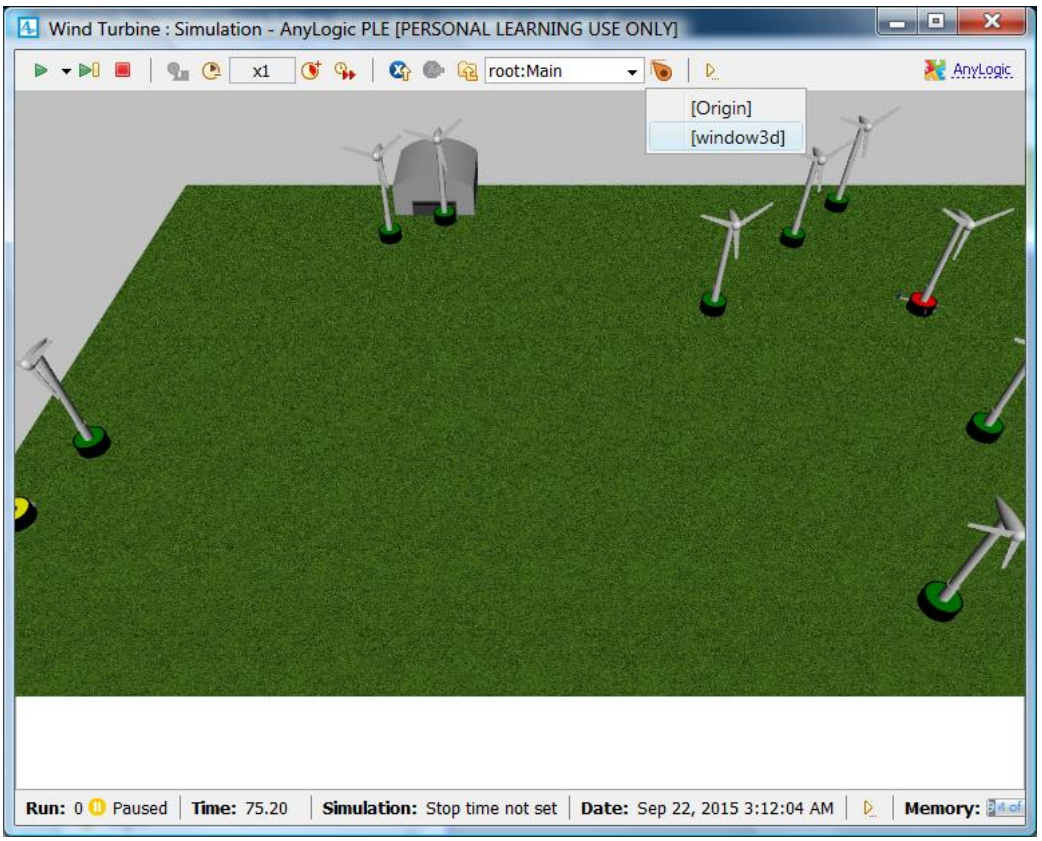
5. In order to enable 3D animation, open the **Presentation** palette and drag **3D Window** from there into the graphical editor. Then you can configure its **Properties**: resize or change color.



6. Click the **Run** toolbar button and run the model.



7. Then click **Navigate to view area...** button and select [**window3d**].

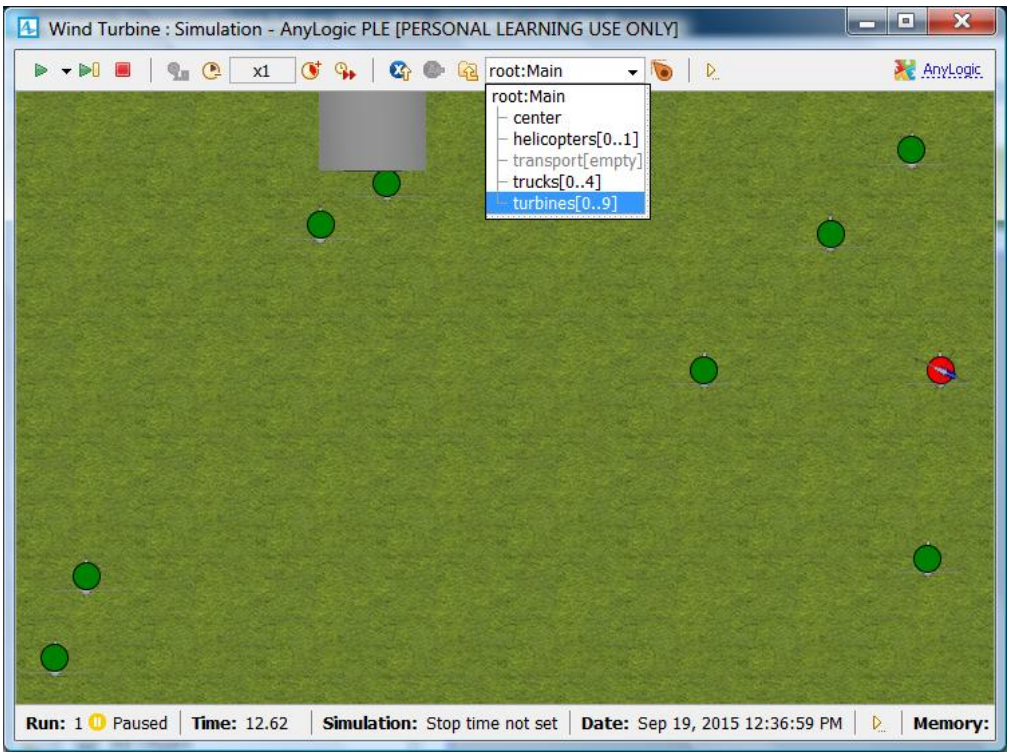


8. Navigate through the 3D scene using the commands described below:

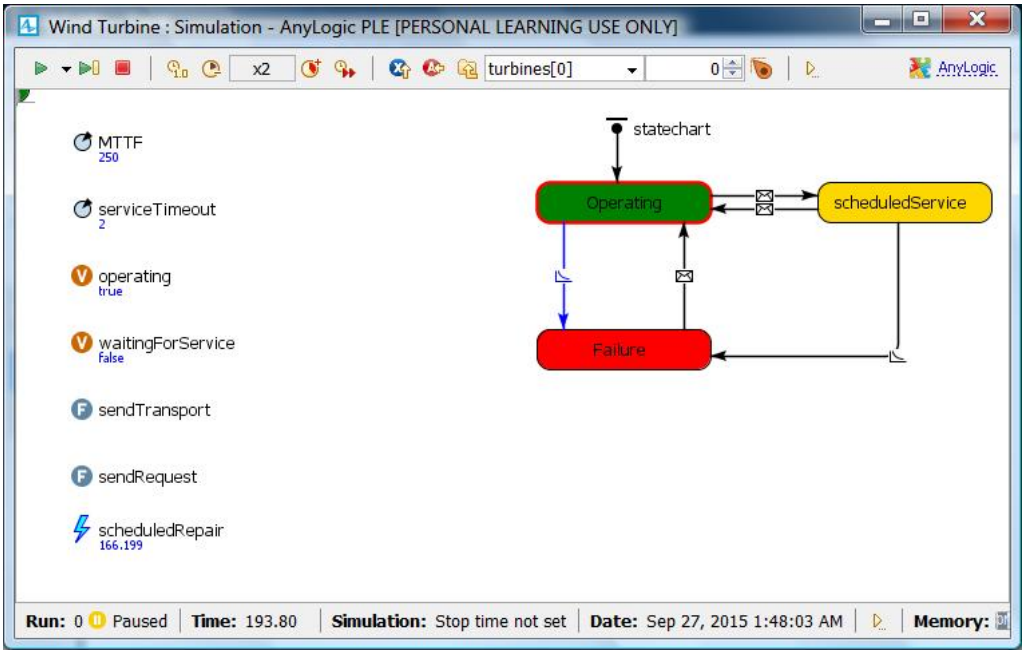
   - Move the scene
     - Press the left mouse button in the 3D view and hold the mouse button pressed.
     - Move the mouse in the required direction.
   - Rotate the scene
     - Press Alt key and hold it pressed.
     - Click in the 3D scene window and, while holding Alt and the left mouse button down,
     - Move the mouse in the required rotation direction.
   - Zoom in/out the scene
     - Scroll the mouse wheel in the 3D window away from / towards you.

9. You can control the running model in several ways: **Pause** or **Stop** the execution, **Speed up** or **Slow down**. You can also observe the agent types using the model explorer.

10. Click the down arrow in model explorer to open the model agent types list. Next to the agent population name you can see the number of agents in it **[..]**. Select, for instance, **turbines [0..9]**. The index starts with 0 - we have ten turbines in this model.

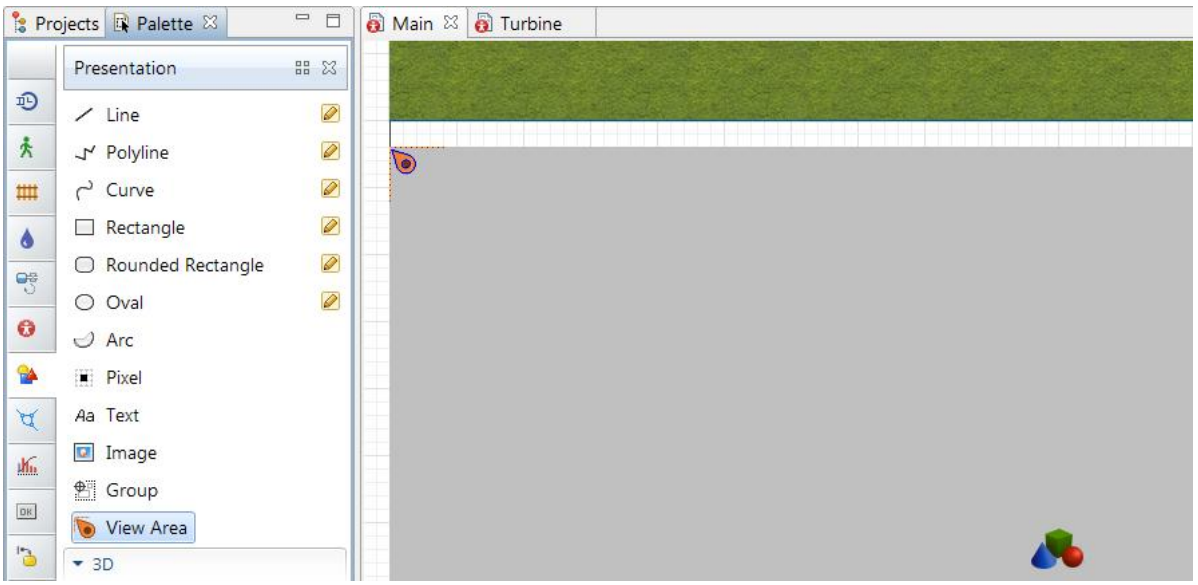


11. You will see *Turbine* and processes going on it. You can check different turbines by selecting the index of a particular turbine. The turbine then appears in sight, its state is highlighted and all agent type elements display the relevant to this turbine information.



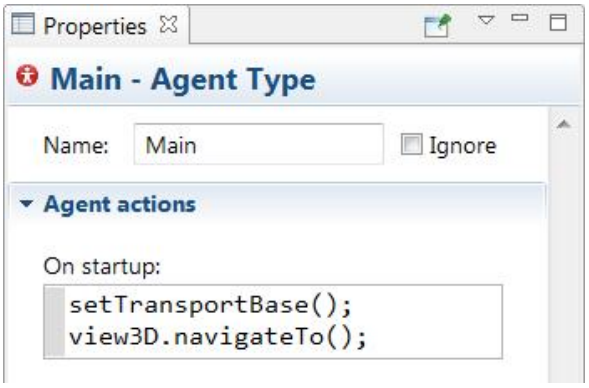
12. In this way you can explore and observe any agent type and agent instance in the model.

13. Create a custom view area in the 3D window. Drag **View area** from the **Presentation** palette and place it in the top left corner of the 3D window.



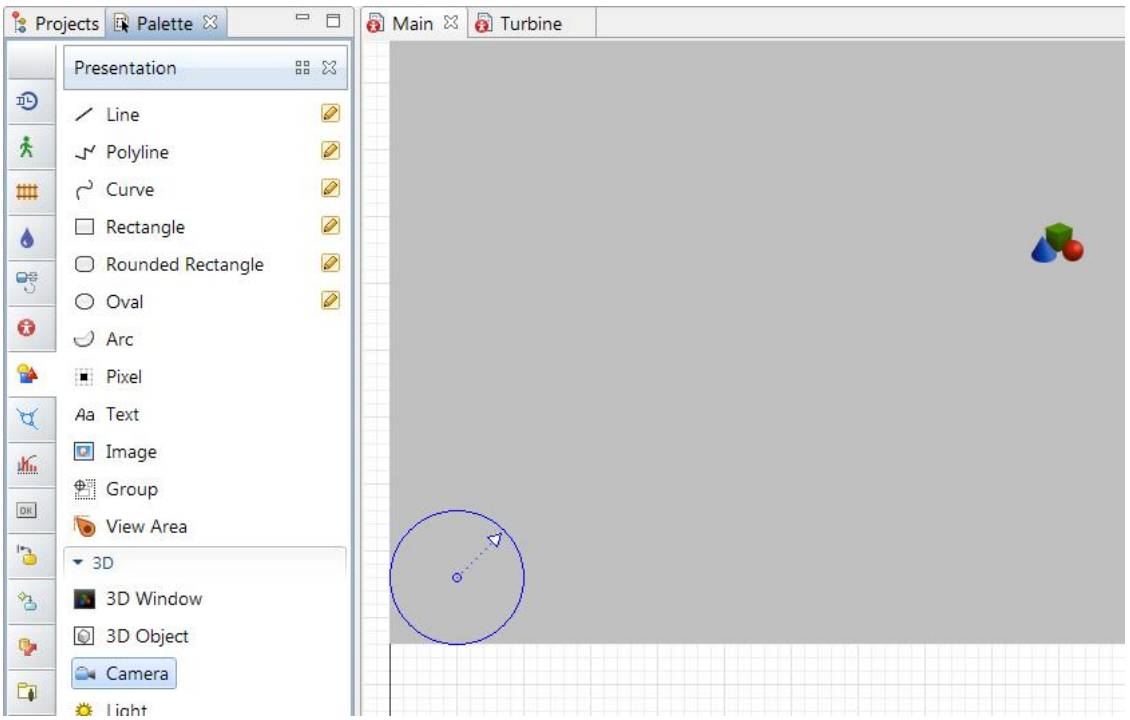
14. Call it *view3D*. We do not need to change any other properties.

15. In order for the 3D scene to open by default when the model is running, change the *Main* **On startup** property. Add there the view navigation command: *view3D.navigateTo();*

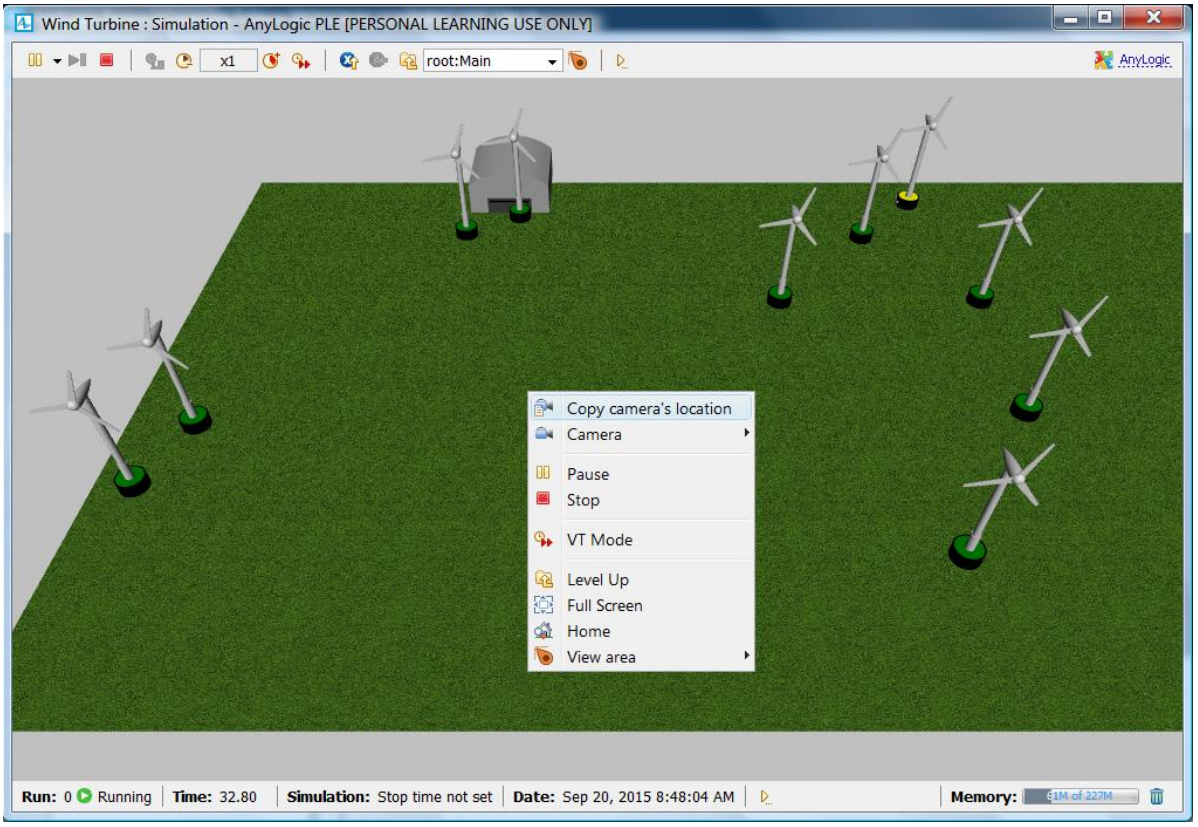


16. If you run the model now you will SE that the 3D animation is presented by default.

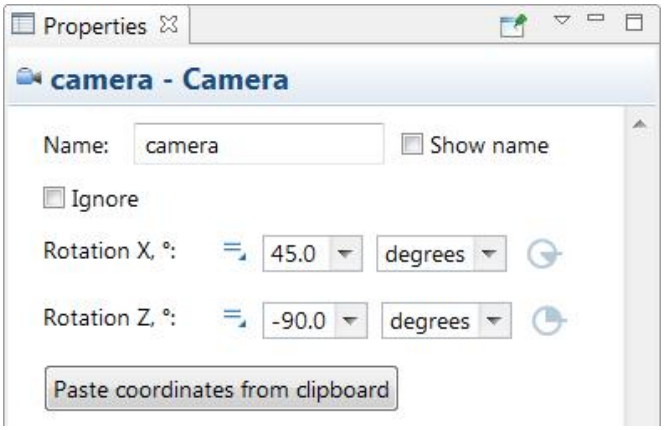
17. Also, you can add **Camera** to the scene to define the specific position of the 3D animation presentation.



18. Drag **Camera** from the **Presentation** palette, select it in the 3D window **Camera** property and run the model. Navigate through the scene and rotate it until you will get the desirable view. Then right-click the canvas and select **Copy camera's location** from the context menu.



19. Go back to the model and **Paste coordinates from clipboard** in the camera properties.



20. Now when you run the model the presentation will open at the selected position in 3D view.